HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



TIỂU LUẬN 01

MÔN HỌC: PHÁT TRIỂN HỆ THỐNG THÔNG MINH

LỊCH SỬ TRÍ TUỆ NHÂN TẠO VÀ THỰC NGHIỆM HỌC MÁY, CNN, RNN

Lớp D23CTPM01

Giảng viên: Trần Đình Quế

Họ và Tên	**Dương Trọng Thắng**
Mã sinh viên	**B23DCCN752**

Hà Nội – 2026

Mục lục

1 Lịch sử phát triển của trí tuệ nhân tạo **1**

1.1 Từ câu hỏi triết học đến bài toán tính toán 1

1.2 Dartmouth và sự ra đời của một lĩnh vực 1

1.3 AI biểu tượng: sức mạnh và giới hạn 2

1.4 Các thời kỳ suy giảm kỳ vọng 3

1.5 Từ quy tắc viết tay sang học máy thống kê 3

1.6 Mạng nơ-ron và học sâu trở lại 4

1.7 Từ mô hình chuyên biệt đến mô hình nền tảng 4

1.8 Những dòng nghiên cứu song song 5

1.9 Dữ liệu, phép đo và khả năng tái lập 5

1.10 Các bước chuyển trong thị giác và ngôn ngữ 6

1.11 AI hiện đại như một hệ thống hoàn chỉnh 7

2 Các kỹ thuật học máy cơ bản **9**

2.1 Bài toán, quy trình và nguyên tắc so sánh 9

2.1.1 Các họ mô hình cơ bản 9

2.1.2 Metric theo bản chất nhiệm vụ 10

2.2 Customer comments: phân loại nhận xét mua sắm 10

2.3 Diabetes large: phân loại nguy cơ tiểu đường 12

2.4 House price: hồi quy giá bất động sản 17

2.5 Tổng hợp Chương 2 19

3 Mạng nơ-ron tích chập **21**

3.1 Từ ảnh số đến đặc trưng không gian 21

3.1.1 Huấn luyện và các kỹ thuật cải thiện 21

3.1.2 Kích thước feature map và số tham số 22

3.1.3 Công bằng trong benchmark và ý nghĩa thời gian 22

3.2 MNIST: chữ số viết tay 23

3.3 CIFAR-10: ảnh vật thể tự nhiên 26
3.4 Phân tích lỗi và lựa chọn mô hình khi sử dụng 30
3.5 Tổng hợp Chương 3 31

4 Mạng nơ-ron hồi tiếp **32**
4.1 Chuỗi thời gian và trạng thái ẩn 32
4.1.1 Thiết kế dự báo và chống rò rỉ tương lai 32
4.1.2 Ví dụ về cửa sổ, chuẩn hóa và phép đo 33
4.1.3 Đọc biểu đồ dự báo thay vì chỉ đọc bảng 34
4.2 Amazon Stock Price: dự báo giá đóng cửa 34
4.3 Avocado Price: nhiều chuỗi giá theo thị trường 36
4.4 Kiểm tra luồng deploy trên localhost 39
4.5 Giới hạn của hai bài toán dự báo 40
4.6 Tổng hợp Chương 4 và ý nghĩa triển khai 41

Tài liệu tham khảo **42**

Danh mục hình ảnh

2.1 Phân bố nhãn, độ dài nhận xét và các thống kê mô tả của customer comments. . 11
2.2 Các từ xuất hiện nhiều theo lớp nhãn; tần suất từ chỉ mô tả dữ liệu, chưa chứng minh giá trị dự báo. 11
2.3 So sánh các metric phân loại customer comments trong notebook gốc; biểu đồ gốc có thêm Random Forest ngoài ba mô hình được chọn. 12
2.4 Ma trận nhầm lẫn chuẩn hóa của benchmark gốc; đối chiếu ba mô hình trong Bảng 2.2. 13
2.5 Giao diện deploy customer comments với nhận xét đầu vào, mô hình và kết quả dự đoán. 13
2.6 Phân phối biến số trên diabetes large theo nhãn. 14
2.7 Tỷ lệ tiểu đường theo các biến phân loại trong dữ liệu khảo sát. 14
2.8 Biểu đồ metric diabetes large của notebook gốc, gồm thêm Gaussian Naive Bayes; bảng trong tiểu luận chỉ so sánh ba mô hình đã chọn. 15
2.9 Ma trận nhầm lẫn của benchmark gốc; cần đặc biệt quan sát số âm tính giả ở lớp dương. 15
2.10 Giao diện deploy diabetes large với các chỉ số đầu vào, mô hình và kết quả dự đoán. 16
2.11 Khảo sát giá nhà, dữ liệu thiếu và đặc trưng bất động sản. 17
2.12 Phân phối các biến số chính trong mẫu nhà được xử lý. 17
2.13 Biểu đồ so sánh hồi quy của benchmark gốc, có thêm Ridge Regression ngoài ba mô hình của tiểu luận. 18
2.14 Phân tích phần dư của mô hình tốt nhất để nhận diện các miền giá khó dự báo. . 18
2.15 Giao diện deploy house price với thông tin căn nhà, mô hình và giá dự đoán. . . 20

3.1 Ảnh mẫu đại diện cho mười lớp chữ số trong MNIST. 23
3.2 Phân bố nhãn train và test của MNIST. 23
3.3 Loss và accuracy theo epoch của ba CNN trên MNIST. 24
3.4 So sánh Accuracy, Macro-Precision, Macro-Recall và Macro-F1 trên MNIST. . 25
3.5 Ma trận nhầm lẫn chuẩn hóa của ba CNN MNIST. 25

3.6 Giao diện deploy MNIST với chữ số đầu vào, mô hình và kết quả phân loại. . . 26
3.7 Ví dụ ảnh của từng lớp CIFAR-10: cùng một nhãn vẫn có nhiều góc nhìn và nền khác nhau. 27
3.8 Phân bố cân bằng của CIFAR-10 trên train và test. 27
3.9 Đường học của ba CNN trên CIFAR-10; validation cho thấy độ khó lớn hơn MNIST. 28
3.10 So sánh các metric phân loại CIFAR-10 trên cùng full test set. 29
3.11 Ma trận nhầm lẫn theo lớp của ba CNN CIFAR-10. 29
3.12 Giao diện deploy CIFAR-10 với ảnh đầu vào, mô hình và nhãn dự đoán. 30

4.1 Minh họa sự suy giảm gradient khi truyền qua nhiều bước thời gian. 33
4.2 Chuỗi Close, phân phối mức giá, thay đổi ngày và volume trong dữ liệu Amazon. 35
4.3 Rolling statistics, tương quan các cột OHLCV và thay đổi theo năm. 35
4.4 Learning curves, giá thực so với dự báo và residual trên test Amazon. 36
4.5 Giao diện deploy Amazon với chuỗi giá Close, framework và giá phiên kế tiếp dự đoán. 37
4.6 Tổng quan mức giá, sản lượng và khác biệt giữa các nhóm Avocado. 38
4.7 Mẫu hình mùa vụ, tương quan và biến động giá Avocado. 38
4.8 Đường học, actual–predicted và residual của Avocado trong notebook. 39
4.9 Giao diện deploy Avocado với region, type, chuỗi giá, framework và kết quả dự đoán. 40

Danh mục bảng

1.1 Các cột mốc tiêu biểu trong lịch sử AI 2
1.2 Ba cách tiếp cận trong lịch sử AI và câu hỏi đánh giá phù hợp 6

2.1 Ba dataset được dùng trong Chương 2 10
2.2 Ba mô hình được chọn trên customer comments (test set) 11
2.3 Ba mô hình được chọn trên diabetes large (test set) 14
2.4 Ba mô hình được chọn trên house price (test set) 18

3.1 Kích thước và cách chia dữ liệu CNN 22
3.2 Ba CNN trên full test set MNIST 24
3.3 Ba CNN trên full test set CIFAR-10 28

4.1 Nhiệm vụ dự báo và cách chia dữ liệu RNN 33
4.2 Ba mô hình trên test Amazon Stock Price 36
4.3 Ba mô hình trên test Avocado Price 37

Chương 1

Lịch sử phát triển của trí tuệ nhân tạo

1.1 Từ câu hỏi triết học đến bài toán tính toán

Trí tuệ nhân tạo (Artificial Intelligence – AI) hướng tới việc tạo ra hệ thống có khả năng suy luận, học từ kinh nghiệm, nhận biết môi trường và xử lý ngôn ngữ. Quá trình phát triển của AI không diễn ra đều đặn: trọng tâm từng chuyển từ các luật được lập trình thủ công sang mô hình xác suất, tối ưu dựa trên dữ liệu và gần đây là các mô hình quy mô lớn tự học biểu diễn. Việc nhìn lại những lần thay đổi quan điểm này giúp giải thích vì sao ML, CNN và RNN được lựa chọn cho các nhóm dữ liệu khác nhau trong những chương sau.

Năm 1950, Alan Turing đề xuất trong bài *Computing Machinery and Intelligence* một phép thử dựa trên đối thoại để thảo luận khả năng biểu hiện hành vi thông minh của máy [1]. Ông không cung cấp một thuật toán duy nhất cho AI. Giá trị của công trình nằm ở việc chuyển một câu hỏi khó định nghĩa, "máy có nghĩ được không", thành một tình huống có thể quan sát và tranh luận. Phép thử cũng bộc lộ giới hạn: biểu hiện ngôn ngữ giống con người không tự động chứng minh máy hiểu thế giới hay hành động đáng tin cậy.

Trước khi có tên gọi AI, nhiều nền tảng đã hình thành. Logic hình thức cho phép viết suy luận thành các bước có thể kiểm tra; lý thuyết tính toán làm rõ lớp bài toán máy có thể thực hiện; điều khiển học nghiên cứu phản hồi và thích nghi. Ý tưởng mạng nơ-ron ban đầu lấy cảm hứng từ hệ thần kinh, còn thống kê cung cấp cách xử lý bất định. Những dòng nghiên cứu này có điểm chung là xem "trí tuệ" như một quá trình có cấu trúc, có thể biểu diễn và thử nghiệm, thay vì chỉ là phẩm chất bí ẩn của con người.

1.2 Dartmouth và sự ra đời của một lĩnh vực

Bản đề xuất cho dự án nghiên cứu mùa hè Dartmouth được soạn năm 1955 và triển khai vào năm 1956 đã sử dụng thuật ngữ "artificial intelligence" để gọi tên một chương trình nghiên cứu chung [2]. Giả thuyết trung tâm là các thành phần của học tập và trí thông minh có thể được mô tả đủ rõ để máy tính mô phỏng. Dù chưa chứng minh bài toán đã được giải quyết, sự kiện này đã định hình một lĩnh vực với mục tiêu, thuật ngữ và cộng đồng nghiên cứu riêng.

Trong thời kỳ đầu, máy tính rất hạn chế về bộ nhớ và tốc độ nhưng các bài toán nhỏ lại

hứa hẹn. Chương trình có thể chứng minh định lý, giải trò chơi hoặc thao tác biểu thức ký hiệu. Cách tiếp cận chủ đạo là biểu diễn tri thức bằng ký hiệu rồi tìm kiếm trên các phép biến đổi hợp lệ. Nếu một bài toán có trạng thái ban đầu, đích và tập luật chuyển trạng thái, thuật toán tìm kiếm có thể xây dựng chuỗi hành động dẫn tới lời giải. Thành công trên miền hẹp khiến nhiều người kỳ vọng hệ thống tổng quát sẽ đến nhanh hơn thực tế.

Bảng 1.1: Các cột mốc tiêu biểu trong lịch sử AI

Thời điểm	Cột mốc và ý nghĩa
1950	Turing đặt vấn đề đánh giá hành vi thông minh qua đối thoại.
1955–1956	Đề xuất và hội thảo Dartmouth định danh lĩnh vực AI.
1958	Perceptron thúc đẩy nghiên cứu học từ ví dụ.
1960–1970	Tìm kiếm, suy luận ký hiệu và hệ thống hội thoại trong miền hẹp.
1980	Hệ chuyên gia được triển khai trong một số nghiệp vụ; học bằng lan truyền ngược phát triển.
1990–2000	Học thống kê và dữ liệu quy mô lớn trở thành nền tảng ứng dụng.
2012	CNN sâu tạo bước ngoặt trong phân loại ảnh ImageNet.
2017	Kiến trúc Transformer mở rộng cách học quan hệ trong chuỗi.

1.3 AI biểu tượng: sức mạnh và giới hạn

AI biểu tượng xem trí thông minh như khả năng thao tác các biểu diễn có nghĩa. Một hệ thống luật dạng "nếu–thì" có thể giải thích tại sao nó đi tới một kết luận. Tìm kiếm theo chiều rộng, tìm kiếm theo chiều sâu và các heuristic giúp giải các bài toán lập kế hoạch hoặc trò chơi. Trong những miền có luật rõ ràng, đây là một ưu điểm lớn: kiến thức chuyên gia có thể được mã hóa thành quy tắc và kiểm tra từng bước suy luận.

Vấn đề xuất hiện khi hệ thống phải đối mặt thế giới mở. Một bức ảnh không đi vào máy dưới dạng "con mèo đang ngồi trên ghế" mà dưới dạng hàng nghìn hoặc hàng triệu giá trị điểm ảnh. Một câu nhận xét của khách hàng có thể chứa tiếng lóng, phủ định, lỗi chính tả và hàm ý. Muốn xử lý bằng luật, người thiết kế phải liệt kê khối lượng lớn trường hợp, trong khi ngoại lệ luôn tiếp tục xuất hiện. AI biểu tượng vì vậy gặp nút thắt về thu nhận tri thức và độ bền khi môi trường thay đổi.

Hệ chuyên gia của thập niên 1980 minh họa rõ cả hai mặt. Khi tri thức chuyên môn ổn định và phạm vi hẹp, hệ có thể hỗ trợ quyết định khá hiệu quả. Khi luật cần cập nhật liên tục, các quy tắc phụ thuộc nhau trở nên khó bảo trì. Hệ thống cũng khó định lượng mức độ không chắc chắn của dữ liệu đầu vào. Bài học quan trọng là một biểu diễn minh bạch vẫn cần cơ chế học và kiểm định bằng dữ liệu thực tế. Ngày nay, luật và tri thức cấu trúc không biến mất; chúng thường kết hợp với mô hình thống kê trong các hệ thống hoàn chỉnh.

1.4 Các thời kỳ suy giảm kỳ vọng

Thuật ngữ "AI winter" mô tả những giai đoạn đầu tư và niềm tin vào AI suy giảm sau khi kỳ vọng vượt quá năng lực kỹ thuật. Có nhiều nguyên nhân cùng lúc: phần cứng đắt, bộ nhớ ít, dữ liệu chưa đủ, bài toán thực tế phức tạp hơn bài toán trình diễn và một số hệ thương mại không đạt lợi ích như hứa hẹn. Không nên coi các giai đoạn này là lúc nghiên cứu dừng hẳn. Nhiều ý tưởng nền tảng vẫn tiếp tục được phát triển trong thống kê, nhận dạng mẫu, tối ưu và khoa học nhận thức.

Từ góc nhìn phương pháp, các đợt suy giảm là một lời nhắc về cách đánh giá tiến bộ. Một chương trình thắng trong trò chơi nhỏ không chứng minh nó có năng lực giải quyết mọi vấn đề. Một mô hình đạt điểm cao trên tập kiểm tra quen thuộc cũng chưa chắc hoạt động tốt khi dữ liệu thay đổi. Vì vậy, benchmark phải ghi rõ tập dữ liệu, cách chia train/test, metric và giới hạn ngoại suy. Tinh thần ấy được áp dụng ở ba chương thực nghiệm của tiểu luận này.

1.5 Từ quy tắc viết tay sang học máy thống kê

Học máy (Machine Learning – ML) thay đổi câu hỏi thiết kế: thay vì viết mọi quy tắc, ta xác định một họ mô hình, một mục tiêu tối ưu và dữ liệu ví dụ. Mô hình học tham số để dự đoán nhãn hoặc giá trị chưa biết. Một bài toán giám sát có thể viết thành

$$\hat{\theta} = \arg\min_{\theta} \left[\frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f_\theta(x_i), y_i) + \lambda \Omega(\theta) \right]. \tag{1.1}$$

Ở đây hàm mất mát đo độ sai, còn thành phần điều chuẩn hạn chế việc khớp quá mức vào tập huấn luyện. Tư duy này đưa xác suất và khả năng tổng quát hóa trở thành trung tâm của AI ứng dụng.

Các phương pháp tuyến tính, cây quyết định, Naive Bayes và mô hình hạt nhân phát triển mạnh khi dữ liệu số hóa ngày càng nhiều. Khả năng đánh giá bằng tập dữ liệu tách riêng giúp cộng đồng so sánh phương pháp rõ ràng hơn. Tuy nhiên, mô hình truyền thống thường cần kỹ thuật đặc trưng. Với văn bản, ta phải chuyển từ thành vector như TF-IDF; với bất động sản, ta cần mã hóa địa điểm, diện tích và thời gian; với ảnh, các đặc trưng thiết kế tay thường không nắm được hết cấu trúc. Những khó khăn ấy mở đường cho học biểu diễn bằng mạng nơ-ron.

Một bước chuyển khác là thừa nhận tính bất định. Dự đoán không chỉ là một nhãn; nó có thể là xác suất hoặc khoảng giá trị. Trong bài toán mất cân bằng, ngưỡng ra quyết định quan trọng không kém khả năng xếp hạng. Ví dụ ở dữ liệu tiểu đường, lớp dương chỉ chiếm 8,5%; mô hình đoán mọi người "không bệnh" vẫn đạt 91,5% accuracy nhưng không phát hiện được ca nào. Vì vậy sự trưởng thành của ML gắn với cả lý thuyết mô hình lẫn phương pháp kiểm định.

1.6 Mạng nơ-ron và học sâu trở lại

Perceptron của Rosenblatt là một trong những mô hình học trọng số từ ví dụ sớm và có ảnh hưởng [3]. Một perceptron đơn chỉ tạo ra ranh giới tuyến tính, nên không giải được mọi dạng quan hệ. Khi ghép nhiều tầng phi tuyến, mạng có khả năng biểu diễn phong phú hơn; thách thức là tính gradient và tối ưu hiệu quả. Công trình về lan truyền ngược những năm 1980 giúp huấn luyện mạng nhiều tầng thực tế hơn [4]. Dù vậy, dữ liệu và năng lực tính toán thời kỳ đó vẫn hạn chế việc mở rộng.

Học sâu phát triển mạnh khi ba điều kiện gặp nhau: dữ liệu số lớn, phần cứng tính toán song song và kỹ thuật tối ưu tốt hơn. CNN tận dụng cấu trúc không gian của ảnh bằng bộ lọc chia sẻ trọng số. RNN xử lý tín hiệu tuần tự bằng trạng thái ẩn truyền theo thời gian. Các mô hình này không loại bỏ nhu cầu thiết kế thí nghiệm; chúng chuyển phần việc trích đặc trưng từ người lập trình sang quá trình học tham số. Sự khác biệt giữa dữ liệu ảnh, văn bản và chuỗi thời gian vẫn quyết định kiến trúc phù hợp.

Năm 2012, hệ CNN sâu trong nghiên cứu ImageNet của Krizhevsky và cộng sự cho thấy hiệu quả nổi bật của huấn luyện mạng sâu trên tập ảnh lớn [5]. Cột mốc này thường được nhắc tới vì nó chứng minh học biểu diễn có thể vượt nhiều quy trình đặc trưng thủ công trên một benchmark lớn. Sau đó, kết nối tắt, chuẩn hóa và các kỹ thuật regularization khiến việc huấn luyện mạng sâu ổn định hơn. Thành công của học sâu không nên được hiểu là mọi bài toán đều cần mạng lớn: ở dữ liệu bảng nhỏ hoặc dữ liệu có ít tín hiệu, một cây quyết định hay hồi quy tuyến tính vẫn có thể là lựa chọn tốt hơn.

1.7 Từ mô hình chuyên biệt đến mô hình nền tảng

Trong xử lý chuỗi, RNN và các biến thể LSTM, GRU từng là kiến trúc chủ lực vì có thể cập nhật trạng thái theo từng bước. Hạn chế của chúng là khó song song hóa toàn bộ chuỗi và có thể gặp vấn đề gradient khi phụ thuộc kéo dài. Kiến trúc Transformer năm 2017 dùng attention để học quan hệ giữa các vị trí mà không cần truyền trạng thái tuần tự theo đúng cách của RNN [6]. Từ đó, mô hình ngôn ngữ quy mô lớn trở thành một hướng phát triển quan trọng của AI hiện đại.

Mô hình nền tảng có thể được tiền huấn luyện trên dữ liệu rất lớn rồi thích nghi với nhiều nhiệm vụ. Điều này thay đổi cách xây dựng sản phẩm: thay vì luôn huấn luyện từ đầu, người phát triển có thể dùng mô hình có sẵn, tinh chỉnh hoặc bổ sung dữ liệu bằng truy xuất. Đồng thời, các vấn đề về nguồn dữ liệu, sai lệch, chi phí tính toán, quyền riêng tư và độ tin cậy trở nên rõ nét hơn. Một câu trả lời trôi chảy không đảm bảo đúng sự thật; một dự đoán có xác suất cao trong môi trường thử nghiệm không đảm bảo an toàn ở môi trường vận hành.

Nhìn toàn bộ lịch sử, AI liên tục di chuyển giữa hai yêu cầu: biểu diễn được tri thức và học được từ kinh nghiệm. AI biểu tượng mạnh ở cấu trúc, học thống kê mạnh ở suy luận từ dữ liệu, còn học sâu mạnh ở việc rút trích biểu diễn phức tạp. Ba chương sau đi từ mô hình ML trên bảng và văn bản, qua CNN trên ảnh, đến RNN trên chuỗi thời gian. Mục tiêu không phải

tuyên bố kiến trúc nào luôn tốt nhất, mà là chỉ ra dữ liệu, cách đánh giá và chi phí triển khai quyết định kết luận kỹ thuật như thế nào.

1.8 Những dòng nghiên cứu song song

Lịch sử AI thường được kể như chuỗi các kiến trúc thay thế nhau, nhưng trên thực tế nhiều hướng tồn tại đồng thời. Nghiên cứu tìm kiếm và lập kế hoạch tập trung vào hành động: với một mục tiêu và các phép chuyển trạng thái, máy phải chọn đường đi. Nghiên cứu tri thức tập trung vào nội dung: máy cần biết điều gì về thế giới để suy luận. Nghiên cứu nhận dạng mẫu tập trung vào quan sát: từ tín hiệu có nhiễu, máy phải nhận ra một cấu trúc. Ba câu hỏi ấy không thể giải quyết chỉ bằng một thuật toán. Một robot chẳng hạn cần nhìn thấy vật, suy ra vị trí, lập kế hoạch di chuyển và xử lý thay đổi bất ngờ.

Các hệ dựa trên luật có thể diễn đạt điều kiện nghiệp vụ rõ ràng, nhưng thường gặp khó khi dữ liệu không đầy đủ. Hướng xác suất đưa vào cách nói có điều kiện: một dấu hiệu không làm kết luận chắc chắn, mà thay đổi mức tin tưởng vào các giả thuyết. Cách nhìn này quan trọng đối với chẩn đoán, nhận dạng và dự báo. Nếu hai biến cùng phụ thuộc vào nguyên nhân thứ ba, coi chúng là độc lập có thể dẫn tới bằng chứng bị đếm hai lần. Đó là lý do mô hình xác suất cần được xây dựng và kiểm tra trên dữ liệu, không chỉ dựa vào trực giác về quan hệ giữa các biến.

Trong nghiên cứu mạng nơ-ron, ý tưởng học trọng số cạnh tranh với cách viết luật. Người thiết kế không cần viết quy tắc cho từng pixel của chữ số 8; họ cung cấp ví dụ và một hàm mục tiêu. Đổi lại, kết quả thường khó diễn giải bằng vài luật ngắn. Sự đánh đổi giữa minh bạch và khả năng học từ tín hiệu phức tạp đã tồn tại từ rất sớm. Các hệ thống hiện đại thường phối hợp nhiều cách: bộ lọc quy tắc kiểm tra điều kiện cứng, mô hình học máy ước lượng xác suất, và con người rà soát các trường hợp có rủi ro cao.

Một dòng khác là học tăng cường, nơi tác nhân chọn hành động và nhận phản hồi từ môi trường. Khác bài toán giám sát có đáp án cho từng ví dụ, phản hồi có thể đến trễ sau nhiều hành động. Điều này làm vấn đề thử hay khai thác trở nên trung tâm: tác nhân phải thử phương án mới để học nhưng vẫn muốn tối đa hóa phần thưởng hiện tại. Lập kế hoạch, điều khiển tối ưu và học tăng cường kết nối rất sâu với nhau. Tuy các notebook trong tiểu luận chỉ dùng học có giám sát, sự tồn tại của hướng này cho thấy AI rộng hơn việc phân loại và hồi quy.

1.9 Dữ liệu, phép đo và khả năng tái lập

Từ cuối thế kỷ XX, dữ liệu số ngày càng nhiều đã thay đổi đơn vị tiến bộ của AI. Một phương pháp có thể được đánh giá trên cùng bộ dữ liệu với phương pháp khác; các phép đo thống nhất giúp phát hiện cải thiện thật và cả cải thiện do thay đổi điều kiện thử. Tập train dùng để học, validation để chọn cấu hình, test để đánh giá cuối cùng. Nếu xem test nhiều lần rồi điều chỉnh mô hình, test dần trở thành một phần của quá trình chọn lựa và không còn đại diện cho dữ liệu chưa thấy. Đây là vấn đề phương pháp luận có thể làm sai lệch cả kết luận của một công trình nghiên cứu.

Khả năng tái lập đòi hỏi ghi lại không chỉ tên mô hình mà cả phiên bản dữ liệu, seed, quy trình tiền xử lý, cách chia tập, metric và phần cứng. Chẳng hạn một con số accuracy của CNN là mô tả thiếu: nó không cho biết ảnh thuộc dataset nào, phép augmentation ra sao, có dùng tập test để chọn epoch hay không. Trong bài chuỗi thời gian, tách ngẫu nhiên có thể làm rò rỉ tương lai; trong bài văn bản, fit TF-IDF trước khi tách dữ liệu có thể làm lộ từ vựng của test. Nhiều tiến bộ đáng tin cậy của AI đến từ kỷ luật thí nghiệm chứ không chỉ từ một công thức mới.

Khi dữ liệu lớn lên, vấn đề đại diện cũng nổi bật hơn. Một tập ảnh chụp trong điều kiện sáng tốt có thể không bao phủ ảnh thiếu sáng; một tập nhận xét từ một cửa hàng không phản ánh khách hàng ở quốc gia khác. Sai số trung bình có thể che giấu sai số cao ở một nhóm nhỏ. Vì vậy, bên cạnh metric chung cần xem phân bố dữ liệu, ma trận nhầm lẫn, phần dư và thay đổi theo thời gian. Các hình EDA ở Chương 2–4 có vai trò đó: chúng mô tả môi trường mà con số benchmark có hiệu lực.

Bảng 1.2: Ba cách tiếp cận trong lịch sử AI và câu hỏi đánh giá phù hợp

Cách tiếp cận	Nguồn năng lực chính	Câu hỏi kiểm định
Luật và tìm kiếm	Tri thức viết tay, không gian trạng thái	Luật có bao phủ ngoại lệ và suy luận có đúng không?
Học máy thống kê	Đặc trưng và dữ liệu có nhãn	Mô hình tổng quát hóa ra sao trên dữ liệu chưa thấy?
Học sâu	Biểu diễn học từ dữ liệu lớn	Chất lượng có bền khi đổi miền, chi phí và dữ liệu không?

1.10 Các bước chuyển trong thị giác và ngôn ngữ

Sự phát triển của thị giác máy tính cho thấy rõ quá trình đi từ đặc trưng thiết kế tay sang đặc trưng học được. Trước học sâu, một hệ nhận dạng thường tách việc tạo đặc trưng khỏi bộ phân loại: nhà nghiên cứu chọn phép lọc cạnh, mô tả hình dạng hoặc kết cấu, sau đó đưa vector đặc trưng vào mô hình thống kê. CNN gộp hai bước vào cùng một quá trình tối ưu. Bộ lọc ở tầng đầu và phép phân loại ở tầng cuối được điều chỉnh theo cùng loss. Điều này làm mô hình linh hoạt hơn, nhưng cũng đòi hỏi nhiều ví dụ được gắn nhãn và khả năng tính toán đủ lớn.

Đột phá của CNN trên benchmark ảnh lớn năm 2012 không xuất hiện độc lập. Dữ liệu ảnh quy mô lớn cung cấp đủ biến thể để mạng học biểu diễn có ích; phần cứng song song rút ngắn thời gian thử nghiệm; và kỹ thuật huấn luyện làm giảm khó khăn tối ưu. Sau đó, kiến trúc residual cho phép tín hiệu đi qua đường tắt giữa các tầng, góp phần mở rộng độ sâu mà không khiến việc huấn luyện trở nên quá khó [7]. Điều này không có nghĩa cứ tăng tầng là tốt: nếu dữ liệu ít, nhãn nhiễu hoặc đầu vào đơn giản, mạng lớn có thể tốn tài nguyên mà cải thiện rất nhỏ. Hai dataset ở Chương 3 minh họa điều đó: MNIST dễ hơn nhiều so với CIFAR-10.

Trong ngôn ngữ, các hệ dựa trên luật ngữ pháp và từ điển từng chiếm vị trí quan trọng. Mô hình thống kê sau đó học mẫu từ kho văn bản, nhưng biểu diễn rời rạc khó nắm quan hệ

xa giữa các từ. RNN đem lại cách đọc từng token theo thứ tự, lưu thông tin trong trạng thái ẩn; LSTM và GRU giúp xử lý phụ thuộc dài hơn. Attention đặt câu hỏi khác: khi tạo một đầu ra, mô hình nên nhìn vào vị trí nào của chuỗi đầu vào? Transformer mở rộng ý tưởng này bằng cách dùng attention làm thành phần chủ đạo [6]. Khả năng tính toán song song hơn trong huấn luyện là một lý do khiến hướng này mở rộng quy mô mạnh.

Dù kiến trúc học sâu hiện đại gây chú ý, những bài toán nhỏ vẫn cần phương pháp cơ bản. Một bảng vài chục nghìn dòng có thể được giải tốt bằng cây và hồi quy; một chuỗi giá có thể được dự báo rất mạnh bằng giá gần nhất. Lịch sử kỹ thuật không phải cuộc thay thế hoàn toàn, mà là việc bổ sung công cụ với giả định và chi phí khác nhau. Một người làm AI cần hiểu baseline để biết phần cải thiện của mô hình mới đến từ đâu. Kết luận có giá trị nhất là kết luận có thể kiểm tra: trên dataset nào, dưới cách chia nào, theo metric nào và với chi phí nào.

1.11 AI hiện đại như một hệ thống hoàn chỉnh

Sự dịch chuyển từ phòng thí nghiệm sang sản phẩm diễn ra qua nhiều lớp công nghệ. Ở giai đoạn các hệ chuyên gia phổ biến, phần lớn tri thức nằm trong bộ luật và công việc tốn kém là hỏi chuyên gia, chuẩn hóa câu trả lời rồi kiểm tra xung đột giữa các luật. Khi học máy thống kê phát triển, công việc chuyển sang thu thập nhãn, thiết kế đặc trưng và lựa chọn metric. Với học sâu, một phần thiết kế đặc trưng được tự động hóa nhưng nhu cầu dữ liệu, tính toán và theo dõi chất lượng trở nên lớn hơn. Do đó sự thay đổi của AI không chỉ là danh sách thuật toán mới; nó còn là sự thay đổi cách tổ chức công việc kỹ thuật.

Thành tựu trong từng giai đoạn thường phụ thuộc vào bài toán được chọn để đo. Chứng minh định lý ưu tiên suy luận chính xác theo luật; nhận dạng ảnh ưu tiên khả năng tổng quát hóa trước biến thiên pixel; mô hình ngôn ngữ được đánh giá bằng nhiều dạng nhiệm vụ và tương tác. Một hệ có thể rất mạnh theo phép đo của giai đoạn này nhưng yếu ở phép đo khác. Ví dụ, mô hình ảnh có thể nhận lớp chính xác mà không hiểu quan hệ nhân quả giữa các vật trong ảnh. Mô hình ngôn ngữ có thể viết câu mạch lạc nhưng vẫn đưa ra chi tiết sai. Vì vậy tiến bộ nên được mô tả bằng năng lực cụ thể, điều kiện thử và giới hạn, thay vì một khẳng định chung về trí thông minh.

Sự đa dạng của phần cứng cũng định hình hướng nghiên cứu. Thuật toán có chi phí tính toán quá lớn trong một thời kỳ có thể trở nên khả thi khi bộ xử lý song song và bộ nhớ phát triển. Ngược lại, một mô hình rất lớn có thể không phù hợp thiết bị biên hoặc ứng dụng cần đáp ứng tức thời. Trong sản phẩm, lựa chọn một mô hình nhỏ, đủ chính xác và dễ giám sát đôi khi tốt hơn chạy mô hình đạt benchmark cao nhất. Lịch sử AI vì thế gắn liền với kinh tế của tính toán: dữ liệu nào có thể lưu, mô hình nào có thể huấn luyện và dự báo nào có thể phục vụ ở quy mô thực tế.

Ngày nay, một mô hình thường chỉ là một thành phần trong hệ thống AI. Dữ liệu cần được thu thập và làm sạch; kết quả dự đoán cần được trình bày theo cách người sử dụng hiểu; lỗi đầu vào phải được xử lý; phiên bản mô hình phải gắn với đúng phiên bản bộ chuẩn hóa. Nếu một app tải trọng số CNN mới nhưng vẫn giữ mean và standard deviation của bản cũ, kết quả có thể sai dù chương trình chạy không báo lỗi. Câu chuyện này kết nối trực tiếp nghiên cứu thuật

toán với triển khai phần mềm.

Một hệ thống sử dụng AI cần xác định tác động của hai loại lỗi. Trong phân loại nhận xét, bỏ sót review không hài lòng có thể làm mất cơ hội hỗ trợ khách hàng. Trong đánh giá nguy cơ bệnh, bỏ sót một ca dương có hậu quả khác với một cảnh báo sai. Trong giá nhà, sai số lớn ở tài sản đắt tiền làm tăng RMSE đáng kể. Vì thế không có một metric duy nhất đủ để đại diện cho mọi mục tiêu. Việc lựa chọn ngưỡng, hiển thị độ tin cậy và cơ chế con người kiểm tra là quyết định thiết kế gắn với bối cảnh sử dụng.

Lịch sử AI cũng cho thấy không nên đồng nhất sự trôi chảy của giao diện với trí tuệ đáng tin cậy. Hệ ký hiệu có thể giải thích từng luật nhưng thiếu linh hoạt; mạng học sâu linh hoạt nhưng có thể khó dự đoán khi đầu vào lệch khỏi phân bố huấn luyện. Nghiên cứu hiện đại quan tâm nhiều hơn đến đánh giá ngoài miền, theo dõi sau triển khai và hiệu chỉnh mô hình. Đối với sinh viên xây dựng một ứng dụng localhost, bài học thực tế là phải giữ mối liên hệ giữa dataset, checkpoint, bộ tiền xử lý và kết quả giao diện. Mỗi thành phần là một phần của lời giải.

Chương 2

Các kỹ thuật học máy cơ bản

2.1 Bài toán, quy trình và nguyên tắc so sánh

Ba bài toán trong Assignment 03 khác nhau về cả kiểu đầu vào lẫn mục tiêu: văn bản được dùng để phân loại nhận xét, dữ liệu bảng để nhận diện nguy cơ tiểu đường và thông tin bất động sản để hồi quy giá. Chúng vẫn tuân theo một quy trình chung gồm khảo sát dữ liệu, chia tập, fit phép biến đổi trên train, huấn luyện, chọn cấu hình bằng validation và chỉ dùng test cho đánh giá cuối. Các mô hình đại diện từ giả định tuyến tính hoặc xác suất đơn giản đến cây quyết định và MLP phi tuyến.

$$D = D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}}, \qquad D_i \cap D_j = \varnothing \quad (i \neq j). \tag{2.1}$$

Việc chia tập phải hoàn tất trước khi fit scaler, encoder hoặc TF-IDF; nếu không, thống kê từ test có thể ảnh hưởng gián tiếp đến mô hình. Chia phân tầng được dùng cho bài toán lệch lớp để giữ tỷ lệ nhãn giữa các tập tương đối ổn định. Riêng bài toán giá nhà, target được biến đổi log trong lúc học nhưng phải đổi ngược về thang USD trước khi tính các sai số có ý nghĩa thực tế.

2.1.1 Các họ mô hình cơ bản

Logistic Regression ước lượng xác suất lớp dương bằng $p(y = 1 \mid x) = \sigma(w^\top x + b)$, với $\sigma(z) = 1/(1+e^{-z})$. Ranh giới quyết định của mô hình là tuyến tính trong không gian đặc trưng đã mã hóa. Vì vậy nó dễ huấn luyện, dễ giải thích nhưng có thể bỏ lỡ tương tác phi tuyến. Naive Bayes dùng định lý Bayes và giả định các đặc trưng độc lập có điều kiện theo lớp; giả định ấy thường không hoàn toàn đúng, song mô hình đặc biệt hiệu quả với văn bản thưa và số chiều cao.

Decision Tree tạo các phép chia tuần tự theo đặc trưng, hình thành những quy tắc kiểu ngưỡng. Random Forest lấy trung bình nhiều cây huấn luyện trên các mẫu và tập đặc trưng khác nhau để giảm phương sai. MLP học phép biến đổi $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$ qua nhiều tầng. Nếu không có phi tuyến, nhiều tầng tuyến tính chỉ tương đương một phép biến đổi tuyến tính; vì vậy ReLU và các hàm kích hoạt khác là thành phần cốt lõi. MLP trong Assignment 03 được hiện thực bằng NumPy, giúp quan sát trực tiếp forward, backward, mini-batch, điều chuẩn và early stopping.

2.1.2 Metric theo bản chất nhiệm vụ

Với phân loại, accuracy đo tỷ lệ đúng chung nhưng dễ gây hiểu nhầm khi nhãn lệch. Precision trả lời trong các dự đoán dương có bao nhiêu dương thật; recall trả lời trong các dương thật đã tìm ra bao nhiêu. $F_1 = 2PR/(P + R)$ cân bằng hai đại lượng. ROC-AUC và PR-AUC đo khả năng xếp hạng qua nhiều ngưỡng; PR-AUC đặc biệt hữu ích khi lớp dương hiếm. Với hồi quy, MAE diễn giải trực tiếp thành mức sai số trung bình, RMSE phạt mạnh ngoại lệ, RMSLE phản ánh sai số tỷ lệ ở thang log, còn R^2 so với dự đoán trung bình. Không nên so sánh trực tiếp giá trị metric của hai dataset khác đơn vị hoặc khác phân bố.

Bảng 2.1: Ba dataset được dùng trong Chương 2

Dataset	**Dữ liệu gốc**	**Dùng trong notebook**	**Đầu ra**
Customer comments	23.486 review	22.642 review hợp lệ	`Recommended IND`: 0/1
Diabetes large	100.000 người	100.000 người	`diabetes`: 0/1
House price	2.226.382 căn	mẫu 200.000 căn	`price`: giá USD

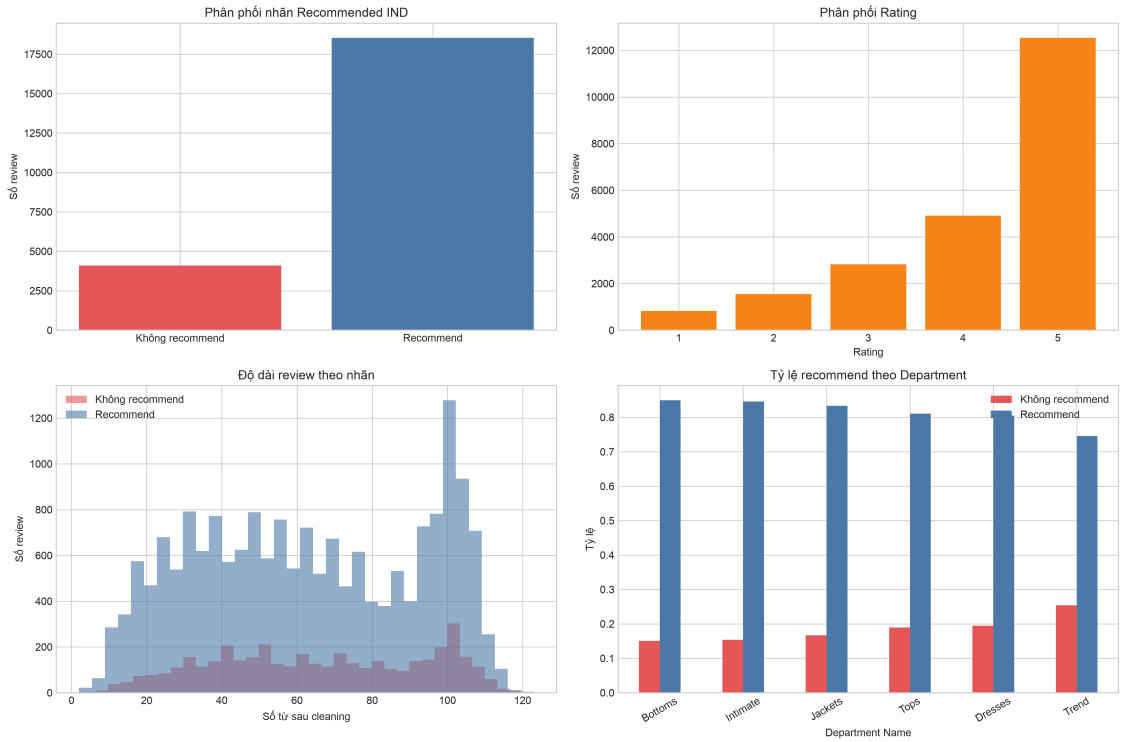
2.2 Customer comments: phân loại nhận xét mua sắm

Nguồn dữ liệu: `https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews`

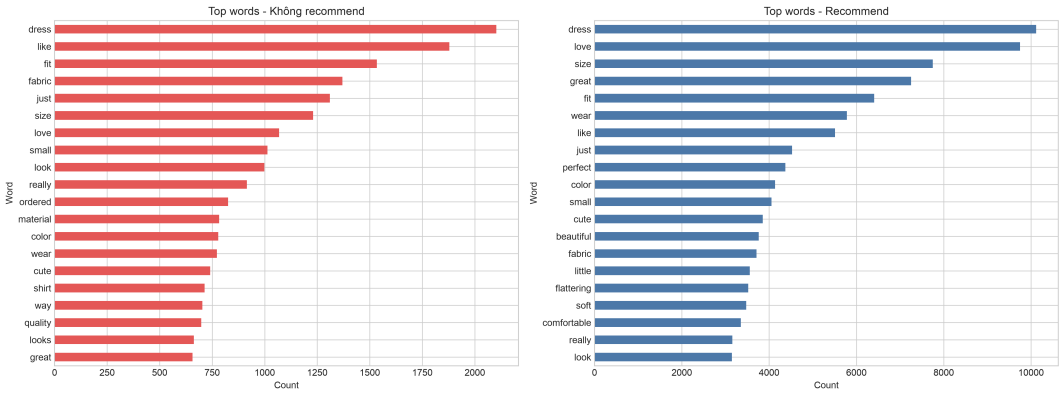
Tệp `Womens Clothing E-Commerce Reviews.csv` có 23.486 dòng. Notebook giữ 22.642 nhận xét hợp lệ sau bước làm sạch văn bản. Cột đầu vào chính là `Review Text`; nhãn là `Recommended IND`. Một dòng mẫu có nội dung “Absolutely wonderful - silky and sexy and comfortable”, điểm đánh giá 4 và nhãn khuyến nghị bằng 1. Nội dung review có thể dài, rất ngắn hoặc trống; vì thế số dòng thật sự đi vào benchmark nhỏ hơn số dòng CSV. Trong dữ liệu gốc có 19.314 nhãn 1 và 4.172 nhãn 0, tương ứng khoảng 82,2% và 17,8%; tỷ lệ sau lọc có thể thay đổi nhẹ.

Văn bản được chuẩn hóa rồi chuyển sang TF-IDF tối đa 1.000 chiều. Với từ t trong tài liệu d, TF-IDF tăng theo tần suất của từ trong d nhưng giảm nếu từ xuất hiện ở nhiều tài liệu. Bộ từ vựng phải được fit trên train; validation và test chỉ được biến đổi bằng bộ từ vựng đã học. Do ma trận này thưa và có nhiều chiều, Multinomial Naive Bayes là baseline phù hợp. Logistic Regression là baseline phân biệt tuyến tính, còn MLP NumPy học thêm tương tác phi tuyến của các đặc trưng văn bản.

Trong ba phương án, Naive Bayes có recall và F_1 cao nhất, tức là phát hiện được gần như toàn bộ review khuyến nghị nhưng chấp nhận nhiều dự đoán dương sai hơn. Logistic Regression phù hợp hơn khi ưu tiên độ chính xác của các review được gắn nhãn tích cực vì precision đạt



Hình 2.1: Phân bố nhãn, độ dài nhận xét và các thống kê mô tả của customer comments.

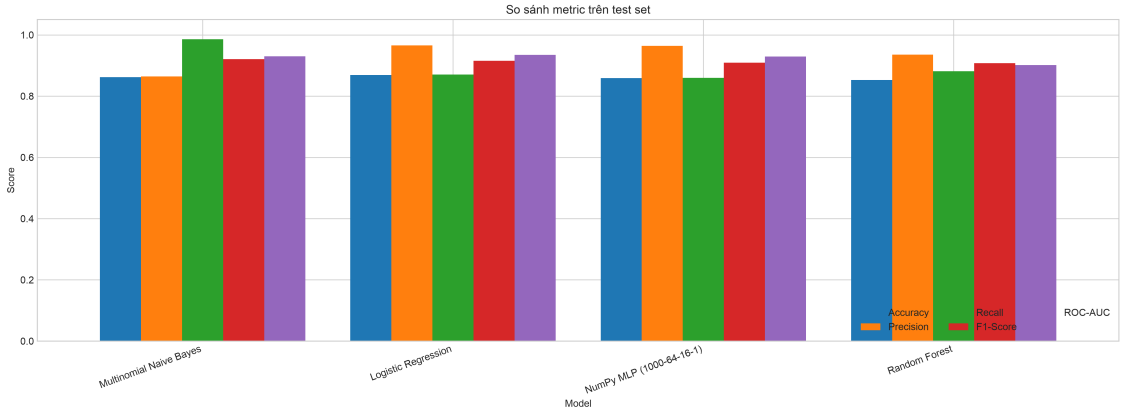


Hình 2.2: Các từ xuất hiện nhiều theo lớp nhãn; tần suất từ chỉ mô tả dữ liệu, chưa chứng minh giá trị dự báo.

Bảng 2.2: Ba mô hình được chọn trên customer comments (test set)

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Train (s)
Multinomial Naive Bayes	0.8618	0.8643	**0.9860**	**0.9212**	0.9299	**0.0020**
Logistic Regression	**0.8686**	**0.9659**	0.8703	0.9156	**0.9348**	0.0222
NumPy MLP	0.8591	0.9640	0.8601	0.9091	0.9297	0.6906

0.9659. MLP không vượt mô hình tuyến tính ở lần chạy này, cho thấy tăng độ phức tạp chưa chắc cải thiện kết quả khi đầu vào là TF-IDF 1.000 chiều.



Hình 2.3: So sánh các metric phân loại customer comments trong notebook gốc; biểu đồ gốc có thêm Random Forest ngoài ba mô hình được chọn.

Ma trận nhầm lẫn nên được đọc theo từng hàng nhãn thật vì lớp khuyến nghị đông hơn nhiều. Độ chính xác tổng thể gần 0.86 có thể che khuất lỗi trên lớp không khuyến nghị. Trong vận hành, nhận diện review không hài lòng thường có giá trị riêng cho chăm sóc khách hàng; vì vậy cần cân nhắc ngưỡng xác suất và mục tiêu nghiệp vụ, thay vì chỉ chọn một con số F_1.

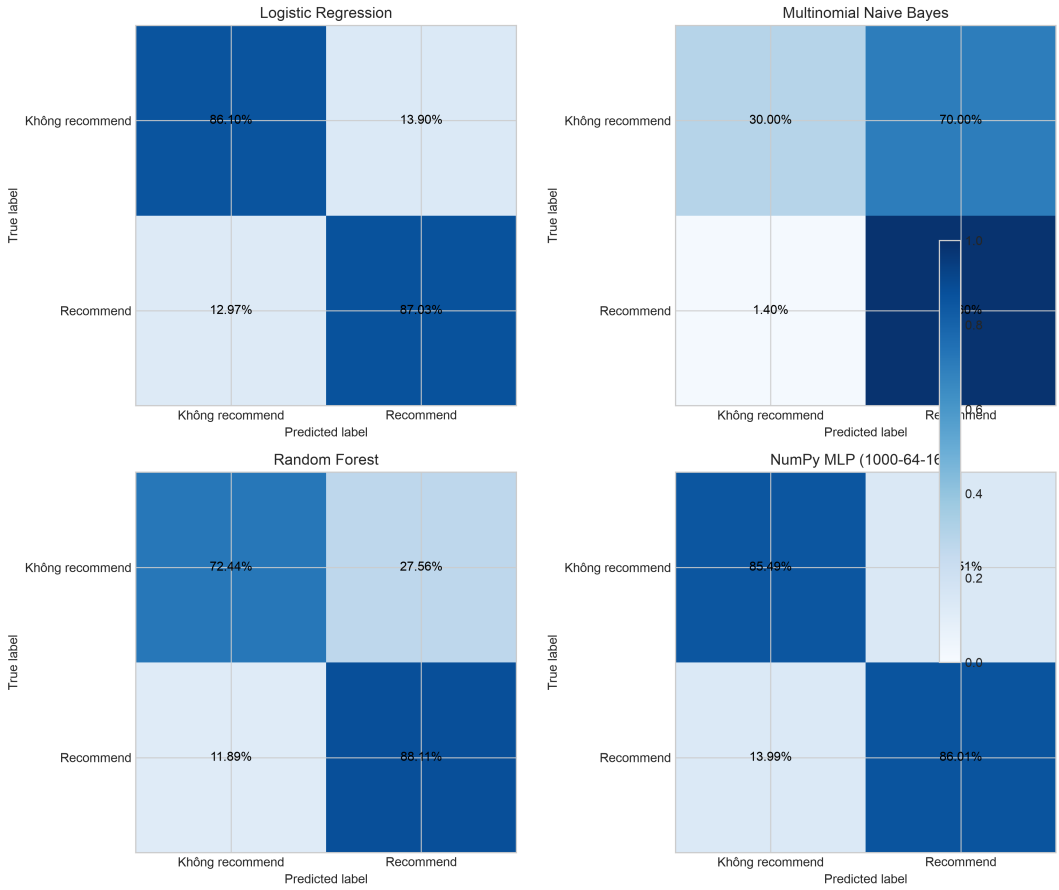
2.3 Diabetes large: phân loại nguy cơ tiểu đường

Nguồn dữ liệu: `https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset`

Tệp `diabetes_prediction_dataset.csv` gồm 100.000 dòng và chín cột: tám biến đầu vào cùng nhãn `diabetes`. Các đặc trưng bao gồm tuổi, BMI, tình trạng tăng huyết áp, bệnh tim, tiền sử hút thuốc, HbA1c và đường huyết. Một mẫu đầu tiên có giới tính nữ, tuổi 80, BMI 25.19, HbA1c 6.6, đường huyết 140 và nhãn 0. Có 8.500 dòng dương (8,5%) và 91.500 dòng âm (91,5%), tạo ra chênh lệch lớp rất lớn.

Notebook one-hot encode các biến phân loại, chuẩn hóa biến số và chia train, validation, test theo tỷ lệ 70/15/15 có phân tầng. Các bước có trạng thái chỉ được fit trên train. Logistic Regression và Decision Tree dùng trọng số lớp cân bằng, trong khi MLP NumPy áp trọng số trong loss; cách này hướng quá trình tối ưu quan tâm hơn tới lớp dương hiếm. Threshold dự đoán được chọn trên validation thay vì cố định 0.5. Sau đó test set độc lập mới được dùng để báo cáo kết quả.

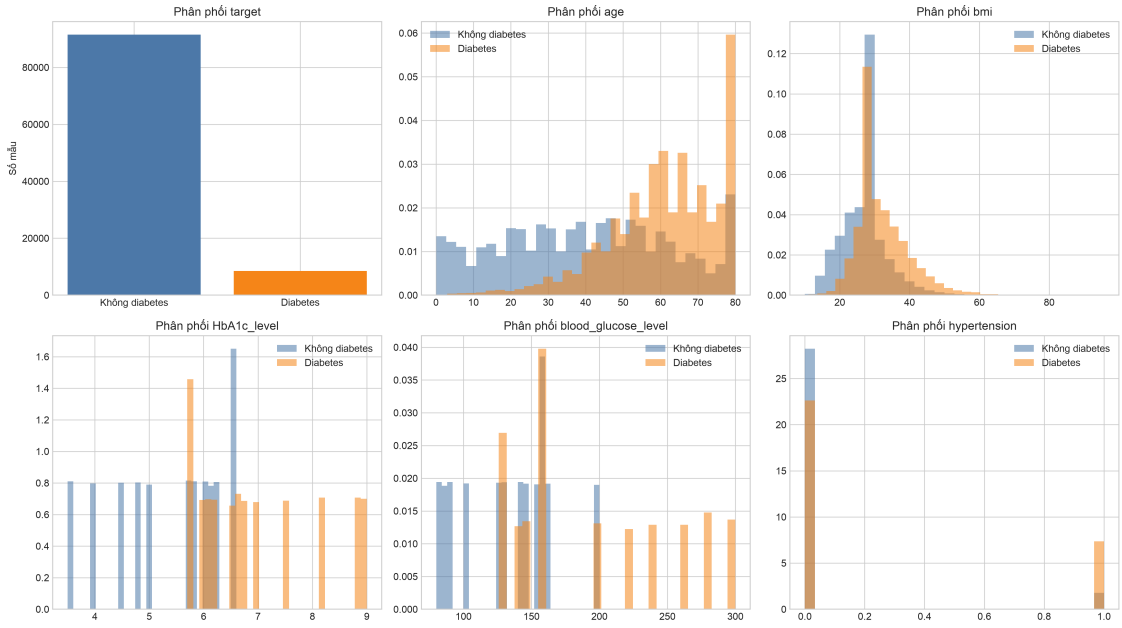
Decision Tree đứng đầu về F_1 và PR-AUC. Giá trị precision 1.0000 tại ngưỡng 0.890 chỉ mô tả tập test hiện tại, không bảo đảm hệ thống thực tế sẽ không tạo dương tính giả. Recall 0.6722 đồng thời cho thấy mô hình vẫn bỏ sót gần một phần ba số ca dương. MLP ưu tiên recall tốt hơn nhưng giảm precision, còn Logistic Regression đạt F_1 gần MLP với thời gian huấn luyện thấp nhất; lựa chọn cuối cùng vì vậy phải dựa trên chi phí của từng loại sai sót.



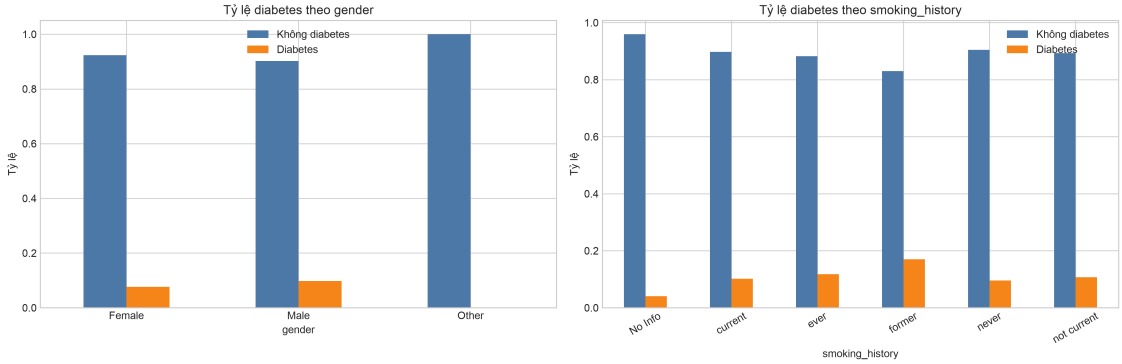
Hình 2.4: Ma trận nhầm lẫn chuẩn hóa của benchmark gốc; đối chiếu ba mô hình trong Bảng 2.2.



Hình 2.5: Giao diện deploy customer comments với nhận xét đầu vào, mô hình và kết quả dự đoán.



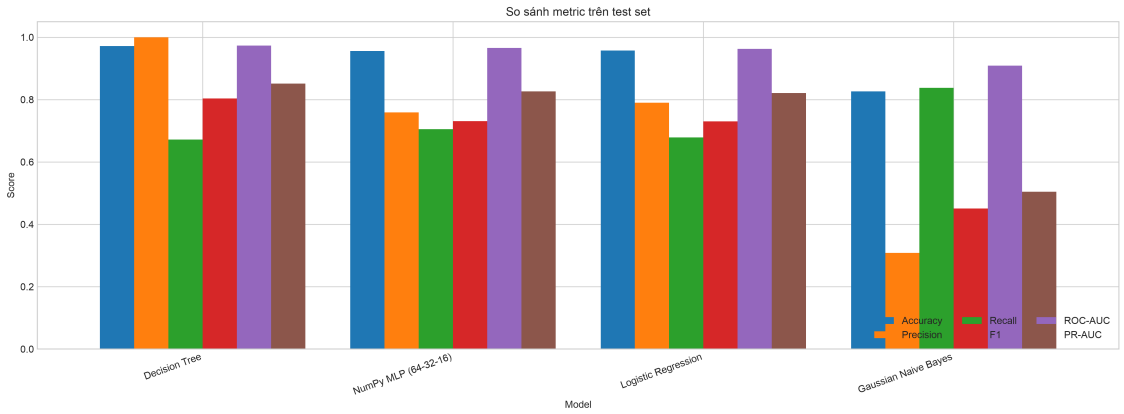
Hình 2.6: Phân phối biến số trên diabetes large theo nhãn.



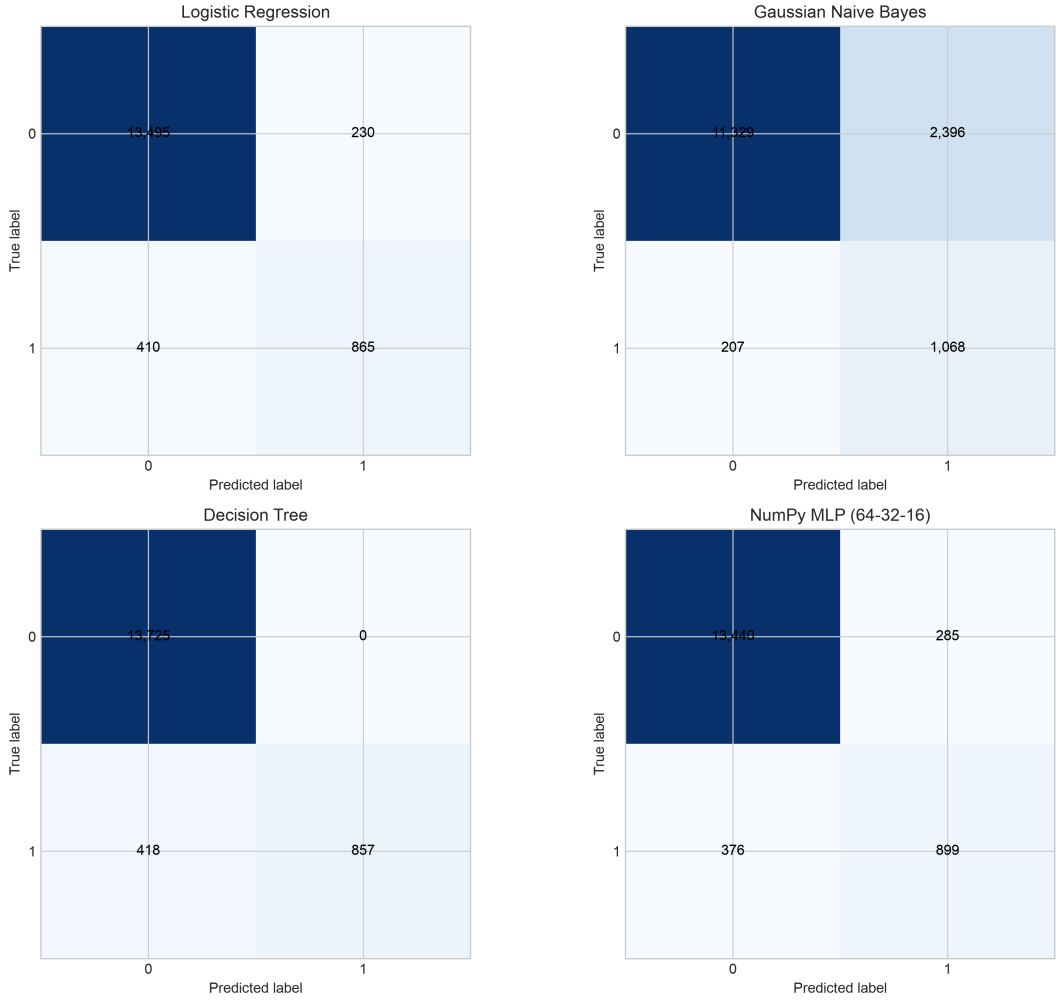
Hình 2.7: Tỷ lệ tiểu đường theo các biến phân loại trong dữ liệu khảo sát.

Bảng 2.3: Ba mô hình được chọn trên diabetes large (test set)

Mô hình	Ngưỡng	Precision	Recall	F1	ROC-AUC	PR-AUC	Train (s)
Decision Tree	0.890	**1.0000**	0.6722	**0.8039**	**0.9731**	**0.8514**	0.0635
NumPy MLP (64-32-16)	0.845	0.7593	**0.7051**	0.7312	0.9658	0.8265	17.2372
Logistic Regression	0.875	0.7900	0.6784	0.7300	0.9629	0.8209	**0.0341**

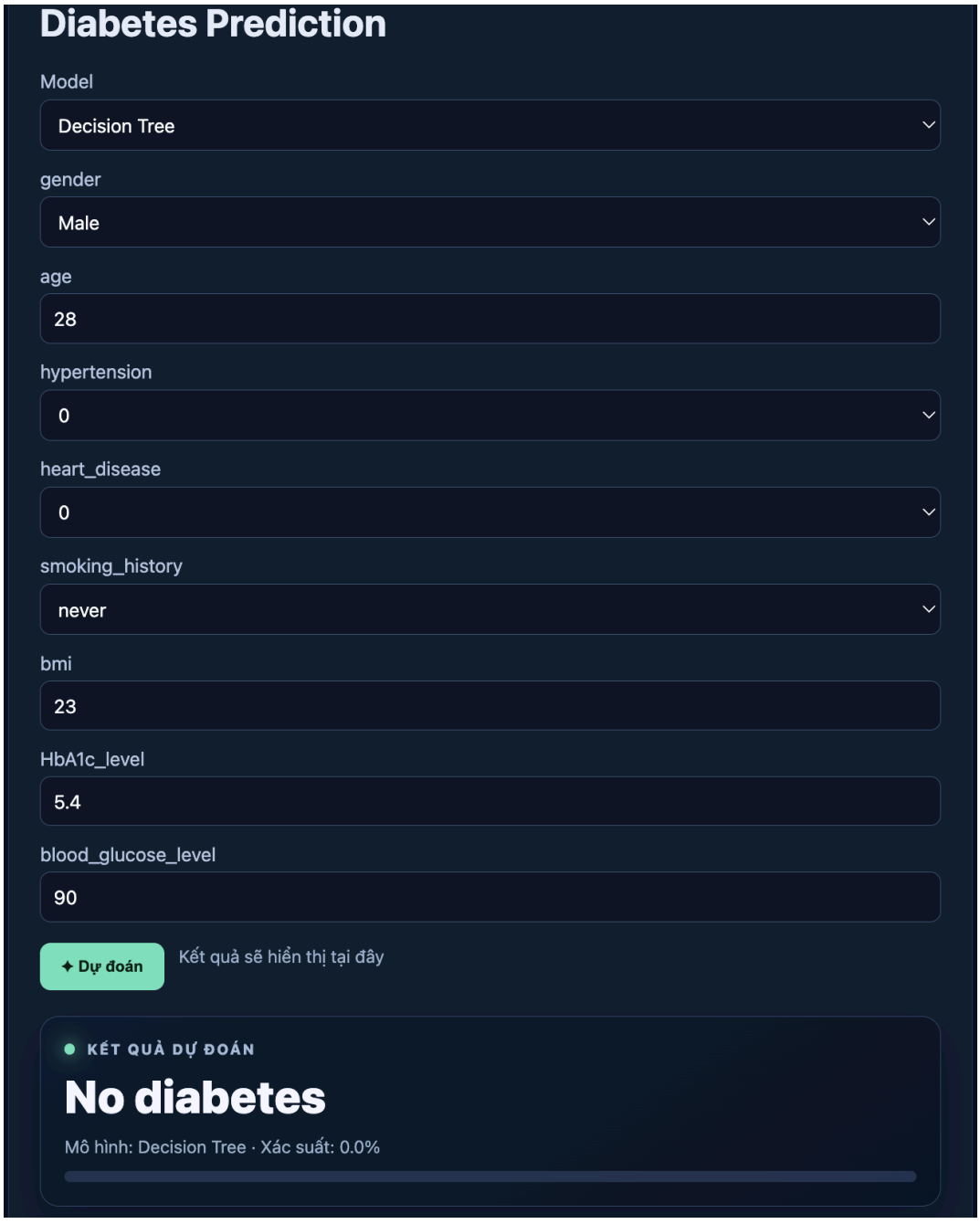


Hình 2.8: Biểu đồ metric diabetes large của notebook gốc, gồm thêm Gaussian Naive Bayes; bảng trong tiểu luận chỉ so sánh ba mô hình đã chọn.



Hình 2.9: Ma trận nhầm lẫn của benchmark gốc; cần đặc biệt quan sát số âm tính giả ở lớp dương.

Về mặt ứng dụng, chỉ số y sinh nhạy cảm với khác biệt giữa quần thể khảo sát và quần thể triển khai. Dataset này dùng cho bài thực nghiệm thuật toán, không phải cơ sở để đưa ra chẩn đoán y khoa. Khi thay đổi tỷ lệ dương ngoài thực tế, precision có thể đổi mạnh dù ROC-AUC gần như giữ nguyên. Cần hiệu chỉnh ngưỡng và kiểm định ngoài mẫu nếu muốn sử dụng trong bối cảnh mới.

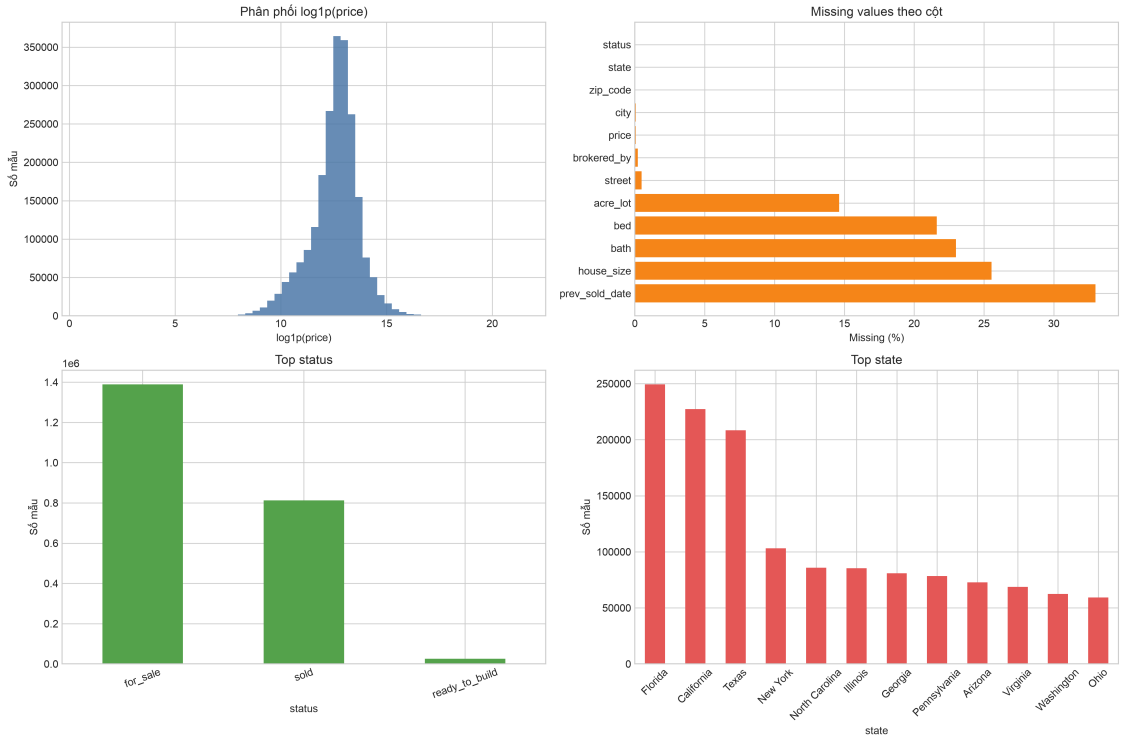


Hình 2.10: Giao diện deploy diabetes large với các chỉ số đầu vào, mô hình và kết quả dự đoán.

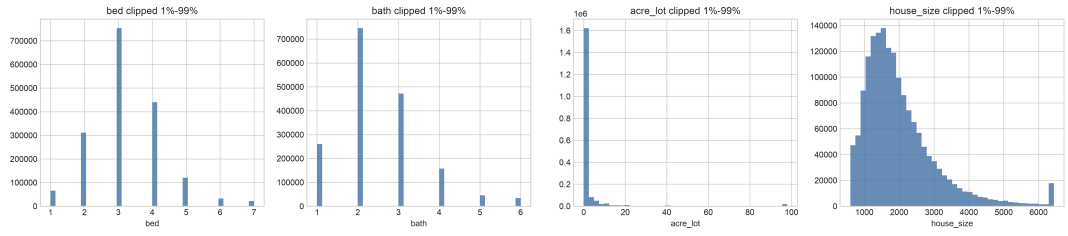
2.4 House price: hồi quy giá bất động sản

Nguồn dữ liệu: `https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset`

Tệp `realtor-data.zip.csv` có 2.226.382 dòng và 12 cột. Notebook lấy mẫu 200.000 dòng để giảm chi phí xử lý và huấn luyện. Target là `price`; các đặc trưng gồm số phòng ngủ, phòng tắm, diện tích nhà, diện tích đất, trạng thái, thành phố, bang, mã ZIP và ngày bán trước. Một dòng mẫu đầu có trạng thái `for_sale`, giá 105.000 USD, ba phòng ngủ, hai phòng tắm, diện tích 920 foot vuông ở Puerto Rico. Trong CSV gốc, khoảng 1,389 triệu dòng có trạng thái `for_sale`, 812 nghìn `sold` và 25 nghìn `ready_to_build`; các nhóm này có thể có ý nghĩa khác nhau đối với giá.



Hình 2.11: Khảo sát giá nhà, dữ liệu thiếu và đặc trưng bất động sản.



Hình 2.12: Phân phối các biến số chính trong mẫu nhà được xử lý.

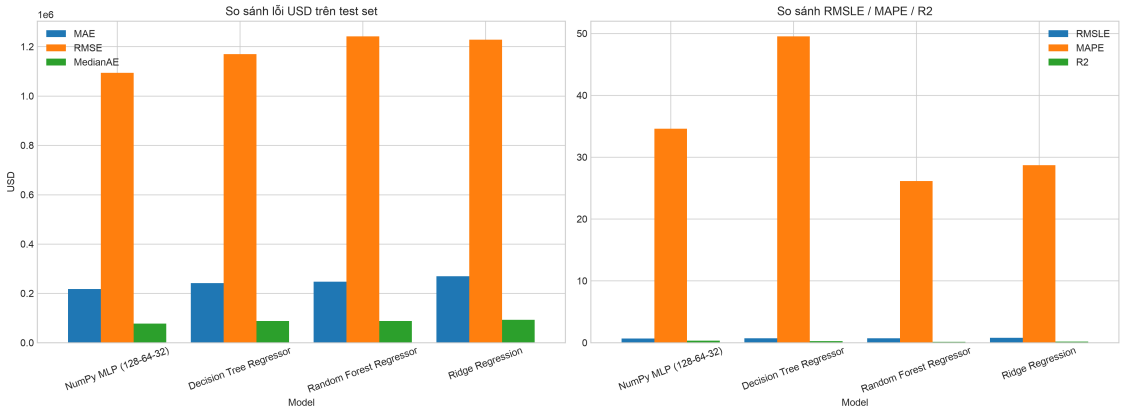
Giá nhà lệch phải và có ngoại lệ lớn, nên notebook học trên $\log(1+\texttt{price})$ rồi đổi ngược về USD khi đánh giá. Các cột phân loại có số giá trị rất lớn như thành phố hoặc ZIP được biểu diễn gọn hơn bằng frequency encoding; thời gian bán trước được tách thành đặc trưng liên quan.

Biến đổi này không nên fit trên toàn bộ dữ liệu trước khi chia, vì thống kê tần suất có thể làm lộ thông tin của test. Mô hình hồi quy phải học đồng thời quan hệ giữa diện tích, vị trí, loại giao dịch và giá trị tài sản.

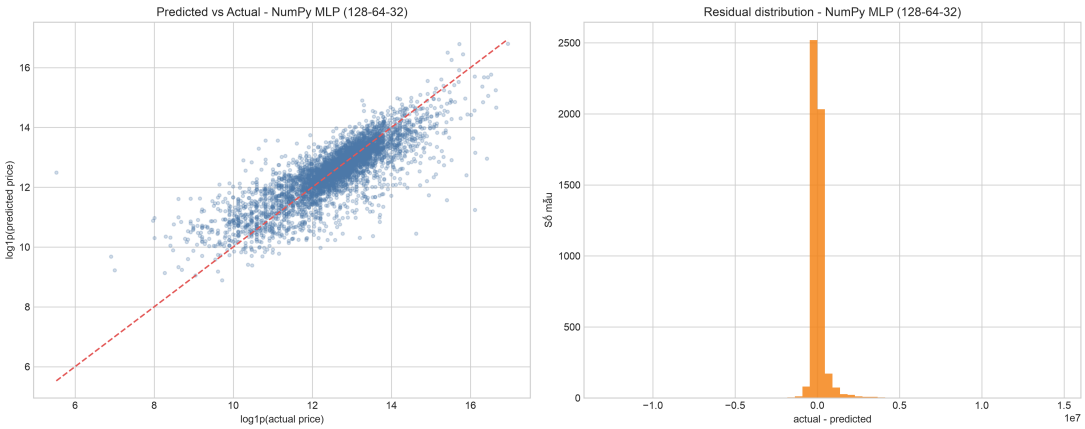
Bảng 2.4: Ba mô hình được chọn trên house price (test set)

Mô hình	MAE (USD)	RMSE (USD)	MedianAE	MAPE	RMSLE	R^2
NumPy MLP (128-64-32)	**217.538**	**1.094.000**	**77.432**	34.6065	**0.6578**	**0.3354**
Decision Tree Regressor	241.108	1.169.300	87.561	49.5228	0.7205	0.2408
Random Forest Regressor	247.668	1.241.500	87.537	**26.1412**	0.7216	0.1442

MLP cho kết quả tốt nhất theo MAE, RMSLE và R^2, nhưng MAE khoảng 217 nghìn USD vẫn đáng kể trong bối cảnh định giá. Khoảng cách lớn giữa MedianAE và MAE, cùng RMSE trên một triệu USD, cho thấy một số bất động sản ngoại lệ tạo ra sai số rất cao. Random Forest lại có MAPE thấp nhất, minh họa rằng thứ hạng mô hình thay đổi tùy mục tiêu ưu tiên sai số tuyệt đối theo USD hay sai số tương đối theo giá trị căn nhà.



Hình 2.13: Biểu đồ so sánh hồi quy của benchmark gốc, có thêm Ridge Regression ngoài ba mô hình của tiểu luận.



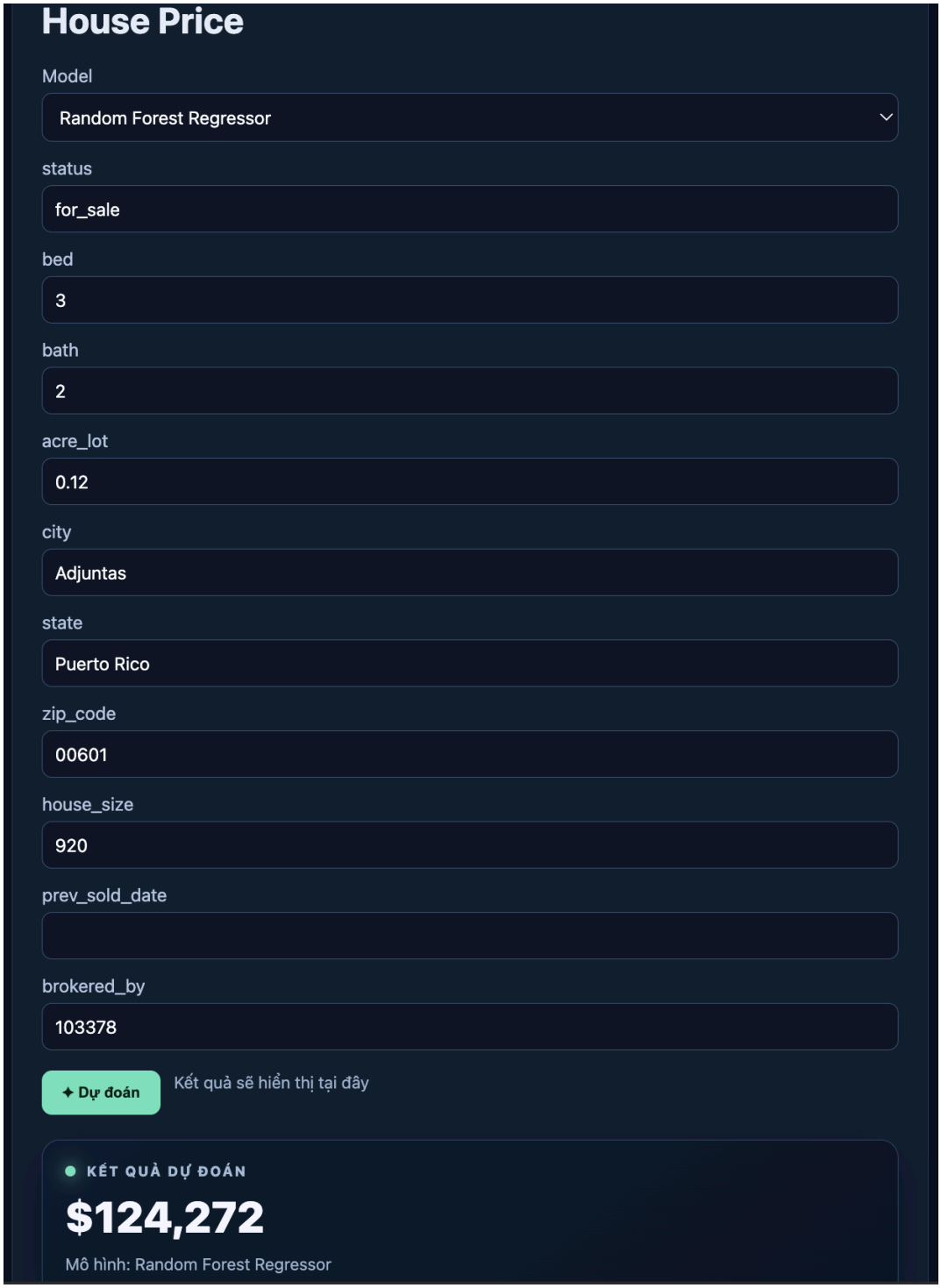
Hình 2.14: Phân tích phần dư của mô hình tốt nhất để nhận diện các miền giá khó dự báo.

Các giá trị lớn và những khu vực hiếm có thể không được biểu diễn đủ trong tập lấy mẫu. Nếu triển khai, cần đánh giá riêng theo vùng địa lý, phân khúc giá và thời gian giao dịch; một

điểm số chung trên tập trộn toàn quốc không cho biết sai số tại từng thị trường. Tập dữ liệu lịch sử cũng có thể lệch với mặt bằng giá hiện tại. Đây là giới hạn quan trọng của mọi dự báo giá nhà dựa trên snapshot dữ liệu.

2.5 Tổng hợp Chương 2

Ba bài toán cho thấy không có mô hình duy nhất dẫn đầu trong mọi hoàn cảnh. Naive Bayes đạt F_1 cao nhất trên TF-IDF nhận xét; Decision Tree nổi bật trên dữ liệu tiểu đường có quy tắc ngưỡng; MLP dẫn đầu về phần lớn metric giá nhà. Các kết quả này là kết quả của pipeline, cách chia dữ liệu và cấu hình notebook hiện tại. So sánh công bằng phải đọc kèm tỷ lệ lớp, số mẫu thực sự dùng, ngưỡng ra quyết định và đơn vị của metric. Việc đưa mô hình lên giao diện localhost giúp kiểm tra luồng inference, nhưng không thay thế việc đánh giá thống kê trên test set.



Hình 2.15: Giao diện deploy house price với thông tin căn nhà, mô hình và giá dự đoán.

Chương 3

Mạng nơ-ron tích chập

3.1 Từ ảnh số đến đặc trưng không gian

Ảnh số không chỉ là tập các giá trị pixel mà còn chứa quan hệ không gian giữa các vị trí lân cận. Nếu ảnh $32 \times 32 \times 3$ bị trải thành vector, MLP phải tự học lại cấu trúc cạnh, góc và họa tiết từ đầu. CNN khai thác tính cục bộ bằng một kernel nhỏ trượt trên toàn ảnh và dùng chung trọng số ở nhiều vị trí; nhờ đó mô hình giảm số tham số và có thể nhận ra cùng một mẫu dù nó xuất hiện ở vùng khác nhau. Với đầu vào X và kernel K, feature map được mô tả gần đúng bởi

$$Y_{i,j,c} = \phi\left(b_c + \sum_{u,v,k} K_{u,v,k,c} X_{i+u,j+v,k}\right). \tag{3.1}$$

Những tầng đầu thường phản ứng với cạnh hoặc màu, còn các tầng sâu hơn ghép chúng thành cấu trúc phức tạp. Khi độ sâu tăng, receptive field của một neuron cũng mở rộng. Pooling hoặc convolution có stride lớn làm giảm kích thước không gian, tiết kiệm phép tính và giảm độ nhạy với dịch chuyển nhỏ. Sau cùng, đặc trưng được tổng hợp và chuyển thành phân bố điểm cho các lớp bằng softmax.

3.1.1 Huấn luyện và các kỹ thuật cải thiện

Loss thường dùng cho phân loại đa lớp là cross-entropy $-\sum_c y_c \log \hat{p}_c$. Gradient truyền từ loss qua tầng fully connected, pooling, activation và convolution để cập nhật kernel. ReLU giúp tối ưu dễ hơn sigmoid trong các tầng sâu; Batch Normalization ổn định phân phối kích hoạt; Dropout và weight decay giảm nguy cơ quá khớp. Data augmentation như dịch, crop và lật ảnh tạo biến thể hợp lý mà vẫn giữ nhãn. Với CIFAR-10, residual connection cho phép block học phần hiệu chỉnh $F(x)$ và trả về $F(x) + x$, giúp huấn luyện mạng sâu hơn [7].

Việc so sánh ba cách cài đặt trong Assignment 04 có hai mục đích. NumPy CNN cho thấy các phép tính cơ bản có thể tự cài đặt và kiểm chứng. PyTorch và Keras cho thấy lợi ích của framework, backend tối ưu và kiến trúc lớn hơn. Tuy nhiên đây không phải phép thử tuyệt đối về framework: NumPy dùng ít ảnh train hơn và mạng nhỏ hơn. Do đó khoảng cách metric gồm cả ảnh hưởng của dữ liệu, kiến trúc, augmentation và tối ưu hóa, không thể quy hết cho thư

viện.

Bảng 3.1: Kích thước và cách chia dữ liệu CNN

Dataset	Gốc train/test	Framework train/val	NumPy train/val	Ảnh và lớp
MNIST	60.000/10.000	54.000/6.000	20.000/3.000	$28 \times 28 \times 1$, 10 chữ số
CIFAR-10	50.000/10.000	45.000/5.000	15.000/3.000	$32 \times 32 \times 3$, 10 vật thể

Ảnh được đưa về khoảng $[0, 1]$ rồi chuẩn hóa theo mean và độ lệch chuẩn tính trên framework train set. Validation, test và subset NumPy dùng cùng thống kê đã học. Train/validation được chia có phân tầng, seed 42; test gốc của mỗi dataset giữ nguyên. Các mô hình đều báo cáo trên cùng full test set. Macro-F1 lấy trung bình F_1 từng lớp, để lớp có ít hay nhiều mẫu đều đóng góp như nhau.

3.1.2 Kích thước feature map và số tham số

Để thấy vì sao CNN phù hợp ảnh, xét một tầng convolution nhận ảnh $H \times W \times C_{\text{in}}$, dùng C_{out} filter cỡ $k \times k$. Nếu có bias, số tham số là $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$, không phụ thuộc trực tiếp vào H và W. Một tầng fully connected nối mọi pixel với cùng số neuron đầu ra sẽ cần khoảng $HWC_{\text{in}}C_{\text{out}}$ trọng số. Khi kích thước ảnh tăng, khác biệt này rất lớn. Chia sẻ trọng số cũng buộc một bộ dò cạnh học ở góc trái có thể áp dụng tại góc phải.

Output của phép tích chập phụ thuộc padding và stride: $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$, tương tự cho chiều rộng. Nếu dùng padding để giữ kích thước, các tầng đầu có thể học chi tiết ở độ phân giải gốc; nếu pooling 2×2 sau block, độ rộng và cao giảm một nửa. Qua nhiều block, số kênh thường tăng khi độ phân giải giảm, vì biểu diễn chuyển dần từ vị trí cụ thể sang kiểu đặc trưng. Số tham số ít không có nghĩa chi phí chạy thấp: convolution vẫn cần nhiều phép nhân cộng trên từng vị trí, nên backend tăng tốc và bố trí tensor rất quan trọng.

Trong mạng phân loại, tầng softmax biến vector logit z thành $\hat{p}_c = e^{z_c} / \sum_j e^{z_j}$. Lớp dự đoán là lớp có logit cao nhất; confidence thường được hiển thị là $\max_c \hat{p}_c$. Giá trị này hữu ích cho giao diện nhưng không tự động là xác suất được hiệu chỉnh. Một mô hình có thể rất tự tin khi ảnh đầu vào khác phân bố train, chẳng hạn ảnh trắng đen CIFAR-10, ảnh MNIST viết ngược hoặc ảnh mờ. Khi phân tích lỗi, cần xem cả nhãn sai, confidence và nội dung ảnh.

3.1.3 Công bằng trong benchmark và ý nghĩa thời gian

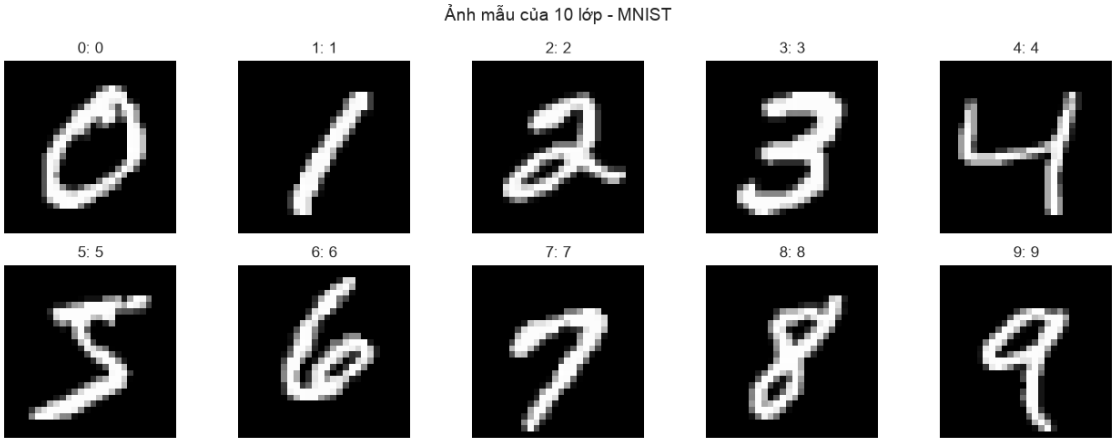
Các mô hình dùng full test set giống nhau nhưng không có cùng ngân sách train. Việc tách train theo tỷ lệ 90/10 cho framework tạo 54.000 ảnh MNIST và 45.000 ảnh CIFAR-10; NumPy dùng subset 20.000 và 15.000 ảnh. Do đó benchmark kiểm tra ba pipeline đã hiện thực, không cô lập tác động của ngôn ngữ lập trình. Nếu muốn trả lời riêng câu hỏi PyTorch và Keras khác nhau về tốc độ bao nhiêu, cần cố định kiến trúc, batch size, thiết bị, precision, số epoch và quy trình đo. Ngay cả khi đó, kết quả chỉ đại diện cho môi trường thử.

Thời gian dự đoán cũng khác thời gian huấn luyện. Một app localhost nhận một ảnh tại một thời điểm chịu chi phí khởi tạo model, chuyển dữ liệu và xử lý HTTP; số đo inference theo batch lớn trong notebook không nói hết độ trễ người dùng cảm nhận. Ngược lại, hệ thống xử lý hàng nghìn ảnh có thể tận dụng batch trên GPU. Vì vậy phần deploy ở mỗi dataset minh họa chức năng suy luận, còn các bảng test mô tả chất lượng thống kê. Hai phép kiểm tra bổ sung cho nhau.

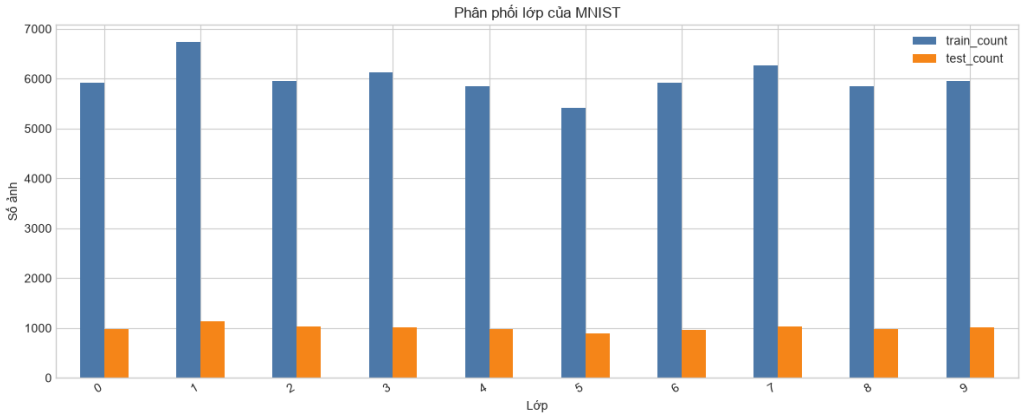
3.2 MNIST: chữ số viết tay

Nguồn dữ liệu: `https://keras.io/api/datasets/mnist/`

MNIST cung cấp 60.000 ảnh train và 10.000 ảnh test, mỗi ảnh là ma trận xám 28×28 mang nhãn từ 0 đến 9. Hình 3.1 cho thấy nét viết, độ nghiêng và độ dày thay đổi đáng kể dù nền ảnh đơn giản. Trong tập train, chữ số 1 có 6.742 mẫu còn chữ số 5 có 5.421 mẫu. Chênh lệch không quá lớn, nhưng Macro-F1 vẫn hữu ích vì đánh giá từng lớp với trọng số ngang nhau thay vì chỉ nhìn accuracy tổng thể.



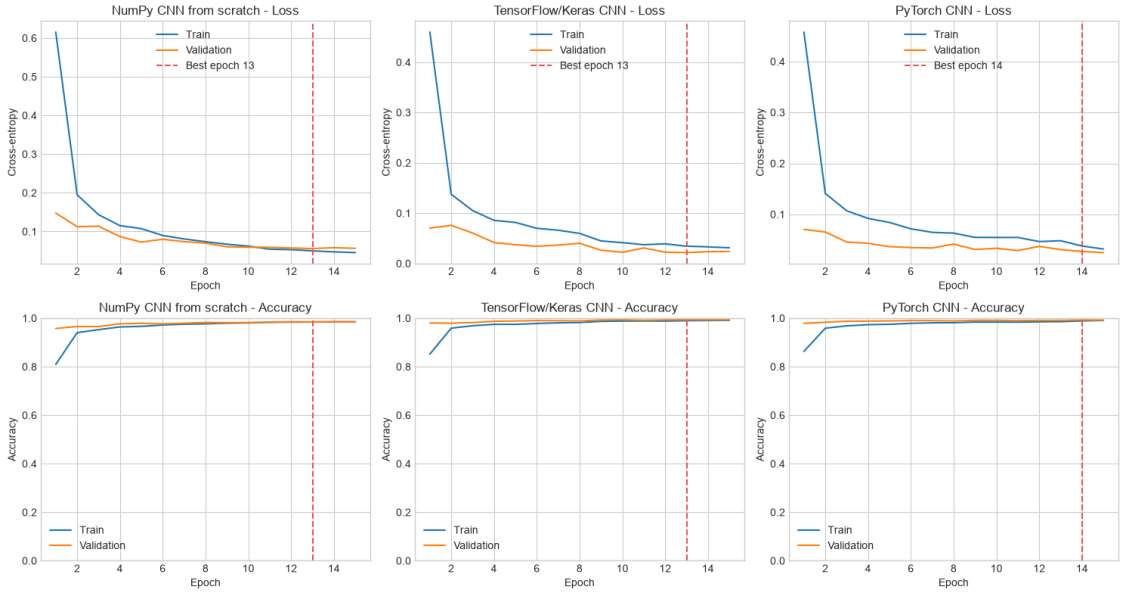
Hình 3.1: Ảnh mẫu đại diện cho mười lớp chữ số trong MNIST.



Hình 3.2: Phân bố nhãn train và test của MNIST.

Một mẫu đầu vào cụ thể là ma trận 784 giá trị pixel; output mong đợi là một nhãn 0–9.

Các giá trị pixel ban đầu từ 0 đến 255. Trong notebook, mean và độ lệch chuẩn học từ train lần lượt khoảng 0.1307 và 0.3082 sau khi chia 255. CNN NumPy dùng hai tầng tích chập với 8 và 16 filter, hai bước pooling và tầng ẩn 64, tổng 52.138 tham số. Phiên bản PyTorch và Keras dùng hai block convolution–BatchNorm–ReLU–pooling, dropout rồi Dense 128; mỗi mạng khoảng 468 nghìn tham số. Chúng cùng loại kiến trúc nhưng không có số tham số hoàn toàn bằng nhau do chi tiết implementation.



Hình 3.3: Loss và accuracy theo epoch của ba CNN trên MNIST.

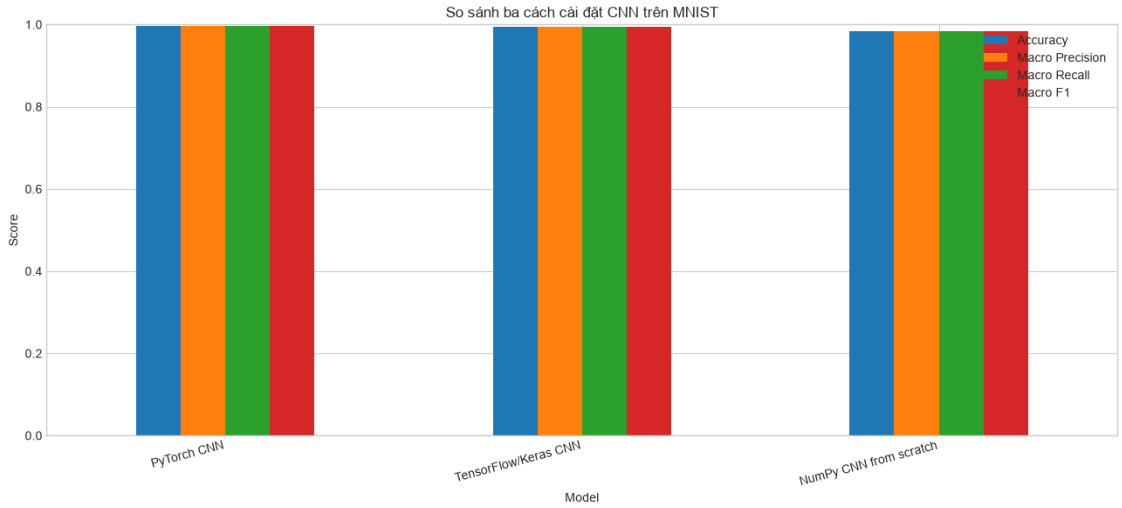
Learning curve cho thấy cả ba mạng hội tụ, nhưng tốc độ và mức cuối khác nhau. CNN NumPy chọn checkpoint ở epoch 13 theo validation loss; PyTorch đạt validation accuracy tốt nhất tại epoch 14, Keras tại epoch 13. Chọn checkpoint bằng validation tránh việc một epoch cuối kém hơn làm lệch phép thử. Tuy nhiên việc dùng tập NumPy nhỏ hơn khiến không thể dùng thời gian huấn luyện để tuyên bố implementation NumPy nhanh hơn một cách tổng quát.

Bảng 3.2: Ba CNN trên full test set MNIST

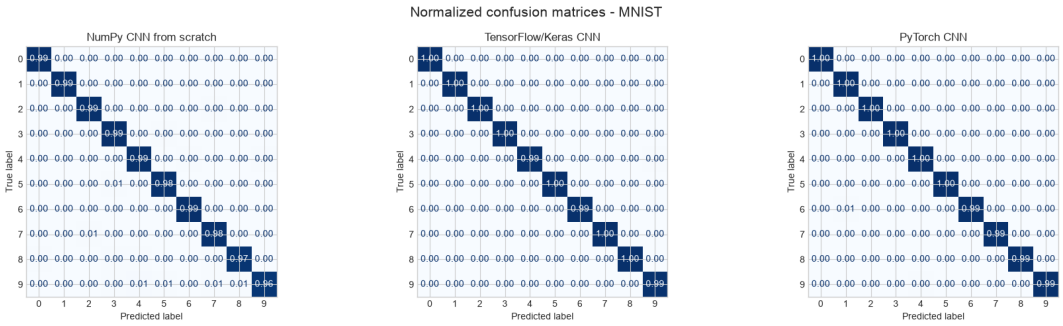
Mô hình	Accuracy	Macro-P	Macro-R	Macro-F1	Tham số	Train (s)
PyTorch CNN	**0.9958**	**0.9959**	**0.9958**	**0.9958**	468.010	90.38
Keras CNN	0.9955	0.9955	0.9955	0.9955	468.394	399.87
NumPy CNN	0.9848	0.9847	0.9847	0.9847	**52.138**	**82.86**

PyTorch cao hơn Keras 0.0003 Macro-F1, một chênh lệch nhỏ trên tập 10.000 ảnh. CNN NumPy thấp hơn khoảng 1,1 điểm phần trăm nhưng dùng ít ảnh train hơn, mạng nhỏ hơn và chỉ có primitive tự cài đặt. Kết quả 0.9848 vẫn xác nhận pipeline convolution/backprop thủ công hoạt động hiệu quả trên dữ liệu đơn giản. Nếu mục tiêu là học thuật toán, NumPy có giá trị giải thích; nếu mục tiêu là độ chính xác cao với công sức triển khai nhỏ hơn, framework phù hợp hơn.

Ma trận nhầm lẫn cho biết cặp chữ số nào dễ bị đánh đồng; ảnh dự đoán sai cần được xem trực tiếp vì một số nét viết vốn mơ hồ ngay cả đối với con người. Độ tin cậy softmax cao



Hình 3.4: So sánh Accuracy, Macro-Precision, Macro-Recall và Macro-F1 trên MNIST.



Hình 3.5: Ma trận nhầm lẫn chuẩn hóa của ba CNN MNIST.

không đồng nghĩa xác suất đã được hiệu chỉnh tốt. Nếu dùng CNN trong hệ nhập liệu thật, ảnh chụp từ điện thoại, giấy có đường kẻ hoặc chữ viết sát mép có thể khác đáng kể với ảnh MNIST chuẩn.



Hình 3.6: Giao diện deploy MNIST với chữ số đầu vào, mô hình và kết quả phân loại.

3.3 CIFAR-10: ảnh vật thể tự nhiên

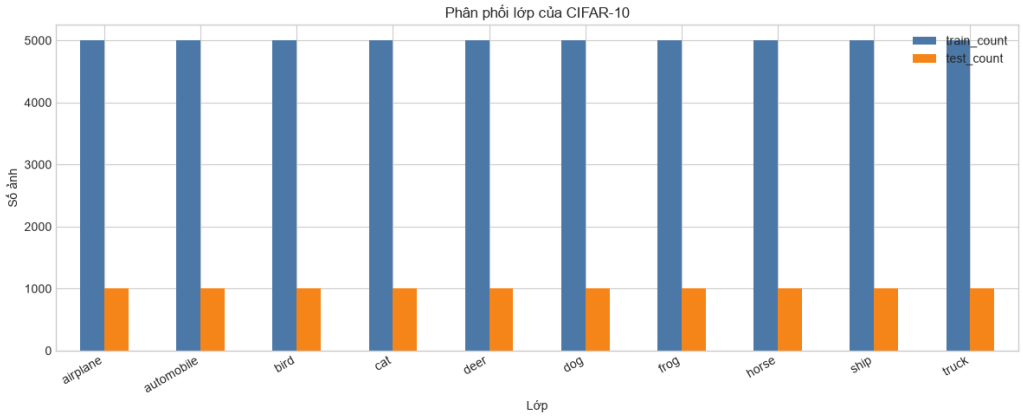
Nguồn dữ liệu: `https://www.cs.toronto.edu/~kriz/cifar.html`

CIFAR-10 có 50.000 ảnh train gốc và 10.000 ảnh test, kích thước $32 \times 32 \times 3$. Mười nhãn là máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu và xe tải. Mỗi lớp có đúng 5.000 ảnh train và 1.000 ảnh test [9]. Một mẫu đầu vào là ảnh màu độ phân giải thấp; output là một trong mười nhãn. Khác MNIST, vật thể có thể nhỏ, bị che, xuất hiện trên nền phức tạp hoặc có hình dạng gần giống lớp khác.

Notebook tách 45.000 train và 5.000 validation cho PyTorch/Keras. NumPy chỉ dùng

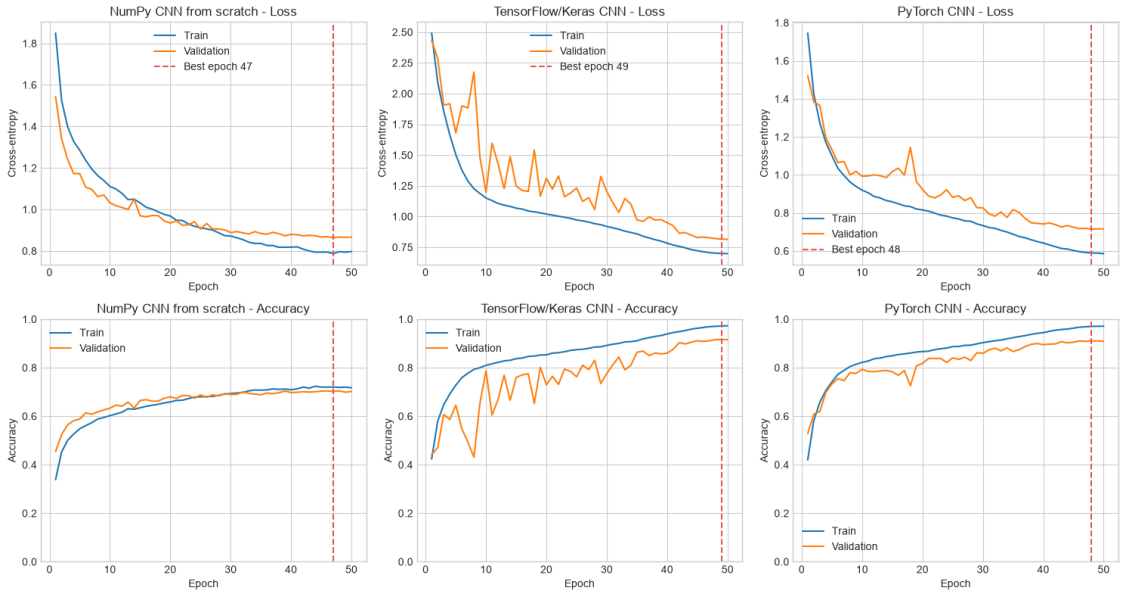


Hình 3.7: Ví dụ ảnh của từng lớp CIFAR-10: cùng một nhãn vẫn có nhiều góc nhìn và nền khác nhau.



Hình 3.8: Phân bố cân bằng của CIFAR-10 trên train và test.

15.000 train và 3.000 validation. Các giá trị trung bình và độ lệch chuẩn của ba channel được fit trên train rồi tái dùng. NumPy CNN có hai block với 12 và 24 filter, tầng ẩn 128 và 200.978 tham số. Hai framework dùng ResNet-20 với ba stage kênh $16 \to 32 \to 64$ và các residual block; số tham số khoảng 272–274 nghìn. Các kỹ thuật crop, lật ngang, label smoothing và lịch learning rate giúp hai mạng framework khai thác dữ liệu phong phú hơn.



Hình 3.9: Đường học của ba CNN trên CIFAR-10; validation cho thấy độ khó lớn hơn MNIST.

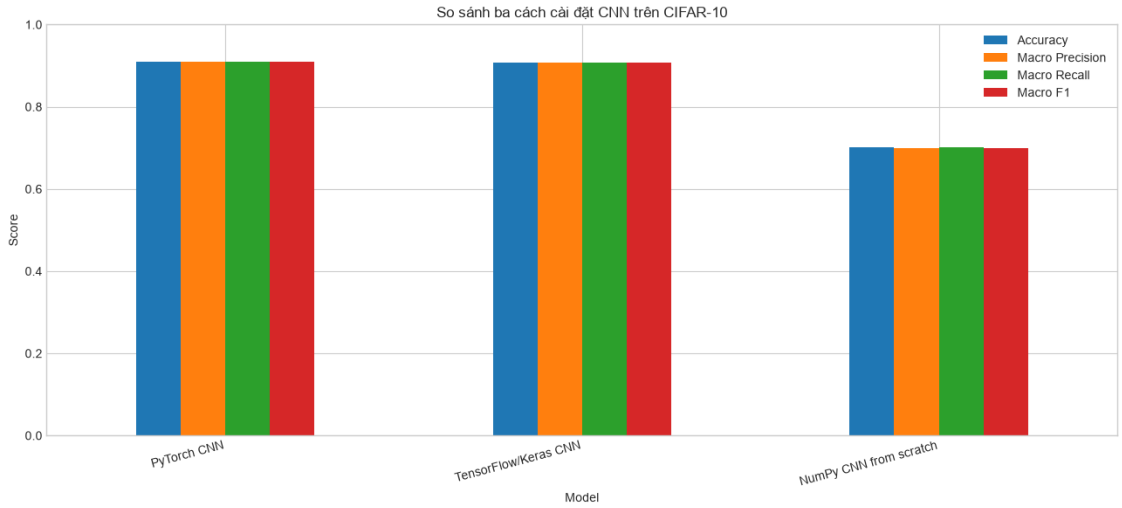
CNN viết bằng NumPy đạt accuracy quanh 0.70, trong khi hai phiên bản ResNet-20 đều vượt 0.90. Khoảng cách này là kết quả tổng hợp của nhiều yếu tố: NumPy dùng ít dữ liệu hơn, mạng nông hơn, không có residual connection và áp dụng chiến lược tối ưu đơn giản hơn. Hai đường train/validation của mô hình NumPy khá gần nhau nên giới hạn chủ yếu có vẻ nằm ở dữ liệu và khả năng biểu diễn hơn là quá khớp nặng. Bài toán ảnh tự nhiên vì vậy thể hiện rõ lợi ích của receptive field lớn và đặc trưng nhiều tầng.

Bảng 3.3: Ba CNN trên full test set CIFAR-10

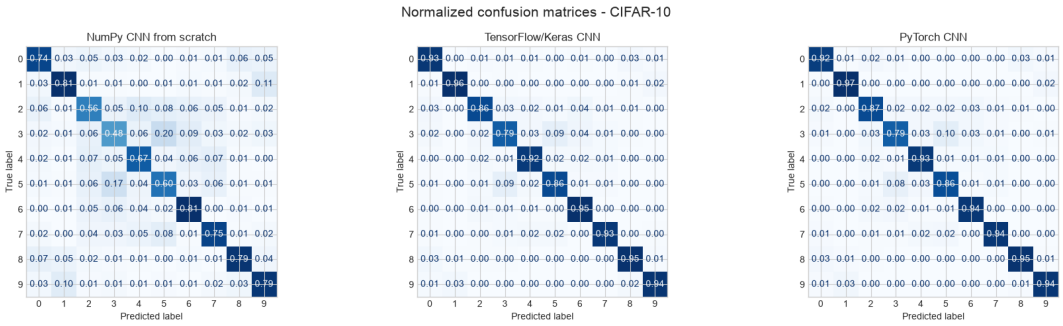
Mô hình	Accuracy	Macro-P	Macro-R	Macro-F1	Tham số	Train (s)
PyTorch CNN	**0.9095**	**0.9092**	**0.9095**	**0.9092**	272.474	822.82
Keras CNN	0.9078	0.9077	0.9078	0.9075	274.042	2636.45
NumPy CNN	0.7012	0.6987	0.7012	0.6994	**200.978**	**501.36**

Hai ResNet-20 gần nhau về chất lượng: PyTorch hơn Keras 0.0017 Macro-F1 trong lần chạy được lưu. Chênh lệch thời gian lớn hơn, song phụ thuộc thiết bị MPS, backend và điều kiện máy tại thời điểm huấn luyện. Không nên suy ra PyTorch luôn nhanh hơn Keras. NumPy thấp hơn rõ, nhưng đạt hơn 70% accuracy trên bài toán khó hơn MNIST; đó là một kết quả phù hợp cho kiến trúc thủ công nhỏ.

Phân tích lỗi theo lớp là cần thiết dù tập cân bằng. Mèo và chó, ô tô và xe tải có cấu trúc ảnh gần nhau hơn các cặp rất khác như tàu và ếch. Mẫu dự đoán sai với confidence cao có thể



Hình 3.10: So sánh các metric phân loại CIFAR-10 trên cùng full test set.



Hình 3.11: Ma trận nhầm lẫn theo lớp của ba CNN CIFAR-10.

là ảnh mờ, vật thể nhỏ hoặc có nhiều vật thể. Khi triển khai ảnh người dùng tải lên, còn cần xử lý resize, kênh màu và chuẩn hóa giống hệt notebook; chỉ cần đổi thứ tự RGB/BGR cũng có thể làm kết quả lệch đáng kể.



Hình 3.12: Giao diện deploy CIFAR-10 với ảnh đầu vào, mô hình và nhãn dự đoán.

3.4 Phân tích lỗi và lựa chọn mô hình khi sử dụng

Đối với một hệ phân loại ảnh, accuracy trung bình chỉ là điểm khởi đầu. Ma trận nhầm lẫn chuẩn hóa cho biết mô hình hay nhầm lớp nào, nhưng mỗi ô chỉ là số lượng tổng hợp. Bước tiếp theo là xem những ảnh sai đại diện: vật thể có bị che khuất không, nhãn có mơ hồ không, ảnh có quá tối hoặc quá nhỏ không. Nếu lỗi tập trung ở một kiểu ảnh, giải pháp có thể là dữ liệu augmentation phù hợp, điều chỉnh cách resize hoặc bổ sung ví dụ, chứ không nhất thiết là tăng số tầng mạng. Tập CIFAR-10 chỉ 32×32 pixel nên nhiều chi tiết nhận dạng vốn đã bị mất trước khi đưa vào mô hình.

Mối quan hệ giữa confidence và độ đúng cũng đáng kiểm tra. Khi softmax cho lớp A giá trị 0.95, người dùng dễ đọc thành xác suất 95% mô hình đúng; cách hiểu đó chỉ hợp lệ khi

mô hình được hiệu chỉnh tốt trên phân bố đầu vào tương ứng. Dữ liệu ngoài miền có thể khiến các logit cực đoan dù dự đoán sai. Một giao diện deploy nên hiển thị nhãn cùng confidence như thông tin hỗ trợ, đồng thời cho phép người dùng kiểm tra lại ảnh đầu vào. Đối với bài MNIST, vùng vẽ hoặc ảnh tải lên cần được xử lý giống tiền xử lý train; đối với CIFAR-10, phải giữ ba kênh màu đúng thứ tự.

Việc chọn mô hình cuối cùng phụ thuộc mục tiêu. PyTorch đứng đầu một chút trên cả hai dataset trong lần chạy đã lưu, nhưng Keras gần như tương đương về Macro-F1. Nếu ứng dụng chỉ cần nhận dạng chữ số ở chất lượng đã đủ, CNN NumPy nhỏ hơn có thể hữu ích cho mục đích giáo dục hoặc môi trường rất giới hạn, miễn là chi phí tự bảo trì phù hợp. Nếu ứng dụng phân loại ảnh vật thể tự nhiên, khoảng cách hơn 20 điểm phần trăm giữa NumPy CNN và hai ResNet-20 là đáng kể. Dù vậy, trước khi triển khai ngoài phạm vi bài tập, cần thử trên ảnh từ đúng nguồn sử dụng, không chỉ trên tập test chuẩn.

3.5 Tổng hợp Chương 3

MNIST và CIFAR-10 cùng có mười lớp nhưng độ khó rất khác nhau. Trên MNIST, CNN nhỏ đã gần đạt trần chất lượng và chênh lệch giữa hai framework rất nhỏ. Trên CIFAR-10, lợi ích của kiến trúc sâu, residual connection và toàn bộ tập train thể hiện rõ hơn. Các bảng benchmark cần được đọc cùng Bảng 3.1: NumPy chỉ dùng subset train, nên ba số trong một hàng không đại diện cho ba mô hình được cung cấp điều kiện huấn luyện giống hệt. Giao diện deploy kiểm chứng đường đi từ ảnh tới nhãn, còn bộ test độc lập mới đo được khả năng tổng quát hóa trong phạm vi dữ liệu chuẩn.

Chương 4

Mạng nơ-ron hồi tiếp

4.1 Chuỗi thời gian và trạng thái ẩn

Chuỗi thời gian khác dữ liệu bảng độc lập vì thứ tự quan sát mang thông tin. Khi dự báo phiên tiếp theo từ 60 phiên trước, việc đảo vị trí các mức giá sẽ làm thay đổi ý nghĩa của mẫu. RNN xử lý tuần tự từng x_t và cập nhật trạng thái ẩn h_t như một bản tóm tắt của lịch sử đã đọc:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \tag{4.1}$$

$$\hat{y}_{t+1} = W_y h_t + b_y. \tag{4.2}$$

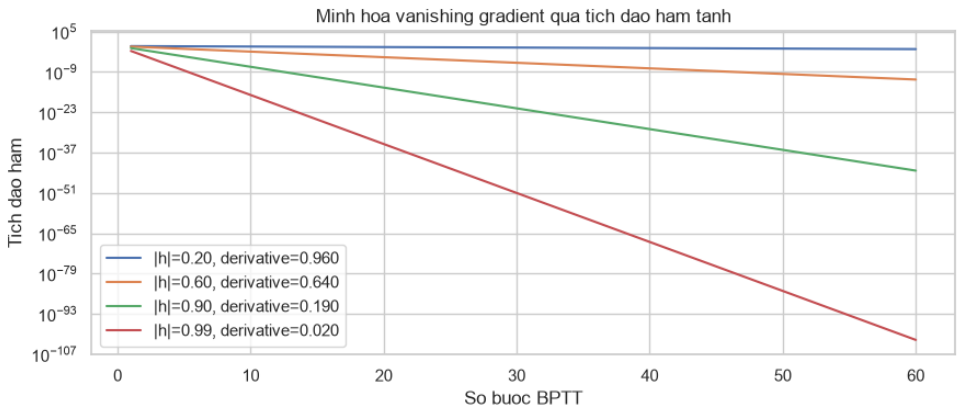
Cùng một bộ trọng số được tái sử dụng ở mỗi bước, vì vậy số tham số không tăng theo chiều dài chuỗi. Assignment 06 dùng cửa sổ giá một chiều làm đầu vào và sinh đúng một dự báo cho thời điểm kế tiếp. Với lookback L, chuỗi $z_1, \ldots, z_T$ được chuyển thành các cặp học $(z_{t-L:t-1}, z_t)$ bằng cơ chế cửa sổ trượt.

RNN được huấn luyện bằng Backpropagation Through Time: mạng được "trải" ra theo số bước L, rồi gradient đi ngược qua các trạng thái. Khi nhân liên tiếp nhiều ma trận Jacobian, gradient có thể nhỏ dần hoặc tăng vọt; đây là khó khăn của phụ thuộc dài hạn [8]. Gradient clipping giới hạn bước cập nhật quá lớn. LSTM và GRU thêm các cổng học cách giữ hoặc quên thông tin, nhưng notebook của bài này dùng vanilla RNN để so sánh hai framework trên cùng kiến trúc. Vì vậy các kết luận thực nghiệm dưới đây không được suy rộng sang LSTM, GRU hay Transformer.

4.1.1 Thiết kế dự báo và chống rò rỉ tương lai

Nếu trộn ngẫu nhiên các cửa sổ, mẫu tương lai có thể đi vào train trong khi cửa sổ quá khứ liên quan lại nằm ở test. Vì vậy notebook giữ nguyên trật tự thời gian và chia xấp xỉ 80% train, 10% validation, 10% test. StandardScaler chỉ được fit trên train; riêng Avocado dùng scaler cho từng chuỗi region–type để không trộn mặt bằng giá giữa các thị trường. Validation phục vụ chọn checkpoint và dừng sớm, còn test chỉ xuất hiện ở bước đánh giá cuối.

Ba mô hình được so sánh là persistence baseline, PyTorch RNN và Keras RNN. Persistence dự báo $\hat{y}_{t+1} = y_t$, nghĩa là giá tiếp theo bằng giá mới nhất trong cửa sổ. Baseline không có



Hình 4.1: Minh họa sự suy giảm gradient khi truyền qua nhiều bước thời gian.

Bảng 4.1: Nhiệm vụ dự báo và cách chia dữ liệu RNN

Dataset	Số dòng	Cửa sổ	Train/Val/Test	Target và bước dự báo
Amazon Stock Price	6.684	60	5.299/662/663	`Close`, phiên giao dịch kế tiếp
Avocado Price	18.249	12	13.498/1.727/1.728	`AveragePrice`, tuần kế tiếp theo nhóm

thời gian train nhưng rất mạnh với chuỗi mức giá biến đổi chậm. Nếu mô hình phức tạp không vượt được baseline này, độ phức tạp tăng thêm chưa đem lại lợi ích dự báo trong phép thử hiện tại. PyTorch và Keras cùng cấu trúc RNN 64 hidden units, activation tanh, dropout 0.2 và đầu ra tuyến tính; tối ưu bằng Adam với MSE loss, early stopping và khôi phục trọng số tốt nhất trên validation.

Các metric được tính sau khi inverse-transform về đơn vị giá gốc. MAE đo sai số tuyệt đối trung bình, RMSE nhấn mạnh dự báo lệch lớn, MAPE là sai số phần trăm trung bình và R^2 so với dự đoán trung bình của test. Với chuỗi giá có xu hướng mạnh, R^2 cao không đảm bảo mô hình dự báo tốt hơn persistence; phải đọc trực tiếp MAE và RMSE của baseline. Các con số thời gian train là số đo của lần chạy notebook, không phải so sánh tuyệt đối giữa hai thư viện.

4.1.2 Ví dụ về cửa sổ, chuẩn hóa và phép đo

Giả sử một chuỗi giá ngắn có các giá trị $[10, 11, 9, 12, 13]$ và lookback bằng 3. Hai cặp học là $([10, 11, 9], 12)$ và $([11, 9, 12], 13)$. Cửa sổ kế tiếp trượt một bước, không bỏ qua thứ tự. Nếu chuẩn hóa bằng mean μ và scale σ học trên train, giá trị đưa vào RNN là $(x - \mu)/\sigma$. Dự báo $\hat{z}$ của mạng phải được đổi ngược thành $\hat{x} = \hat{z}\sigma + \mu$ trước khi so với giá thật. Nếu tính MAE khi một bên ở thang chuẩn hóa còn bên kia ở thang giá, kết quả sẽ vô nghĩa.

Trong chuỗi thời gian, ranh giới train/validation/test có một chi tiết tinh tế. Một cửa sổ test có thể cần vài điểm quá khứ nằm trước ngày bắt đầu test để đủ lookback; việc dùng quá khứ đã biết tại thời điểm dự báo là hợp lệ. Điều không hợp lệ là fit scaler hoặc chọn hyperparameter bằng các giá tương lai của test. Cũng không nên xáo trộn cửa sổ khi xác định split, vì các cửa sổ kề nhau chia sẻ phần lớn đầu vào. Ở Avocado, còn phải giữ nguyên ranh giới nhóm: tuần cuối ở Albany không thể trở thành tuần đầu của một cửa sổ tại Boston.

Persistence baseline có thể được xem là trường hợp đặc biệt của mô hình dự báo không học tham số. Với giá y_t biến động chậm, dự báo $\hat{y}_{t+1} = y_t$ tạo sai số đúng bằng thay đổi một bước $|y_{t+1} - y_t|$. Vì thế độ khó của bài toán phụ thuộc ta dự báo giá mức, sai phân hay phần trăm thay đổi. Một mô hình dự báo mức giá có thể đạt R^2 cao chỉ vì mức giá trên tập test trải rộng; nhưng vẫn thua baseline theo MAE. Đây là lý do Bảng 4.2 không thể bị diễn giải bằng R^2 của RNN tách khỏi hàng persistence.

4.1.3 Đọc biểu đồ dự báo thay vì chỉ đọc bảng

Đường actual–predicted cho thấy mô hình có theo kịp các bước tăng giảm hay bị trễ pha. Residual $e_t = y_t - \hat{y}_t$ cho biết sai số có đổi dấu liên tục hay kéo dài thành một vùng thiên lệch. Nếu nhiều residual dương liên tiếp khi giá tăng, mô hình đang dự báo thấp có hệ thống trong giai đoạn ấy. RMSE sẽ tăng mạnh nếu thỉnh thoảng xuất hiện những đỉnh sai số rất lớn; MAE diễn tả lỗi điển hình ổn định hơn. Biểu đồ theo thời gian vì vậy bổ sung thông tin mà một metric tổng hợp không có.

Phân bố train/test có thể đổi theo thời gian, đặc biệt ở thị trường tài chính. Một mô hình học quan hệ từ nhiều năm cũ rồi dự báo những năm giá ở vùng mới đối mặt với distribution shift. Đánh giá walk-forward sẽ liên tục mở rộng hoặc trượt tập train, dự báo giai đoạn tiếp theo và gom lỗi qua nhiều mốc. Notebook hiện tại dùng một split cố định, nên kết luận chỉ phản ánh đúng split đã lưu. Đề xuất walk-forward là hướng đánh giá tiếp theo chứ không phải thí nghiệm đã hoàn thành.

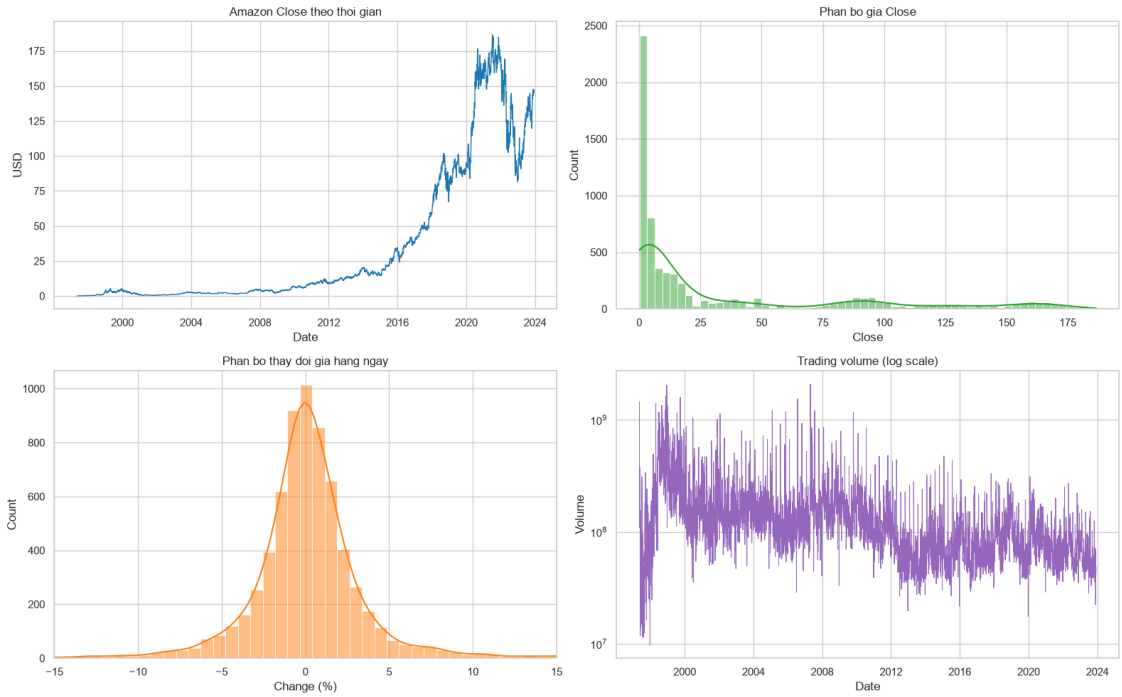
4.2 Amazon Stock Price: dự báo giá đóng cửa

Nguồn dữ liệu: `https://finance.yahoo.com/quote/AMZN/history/`

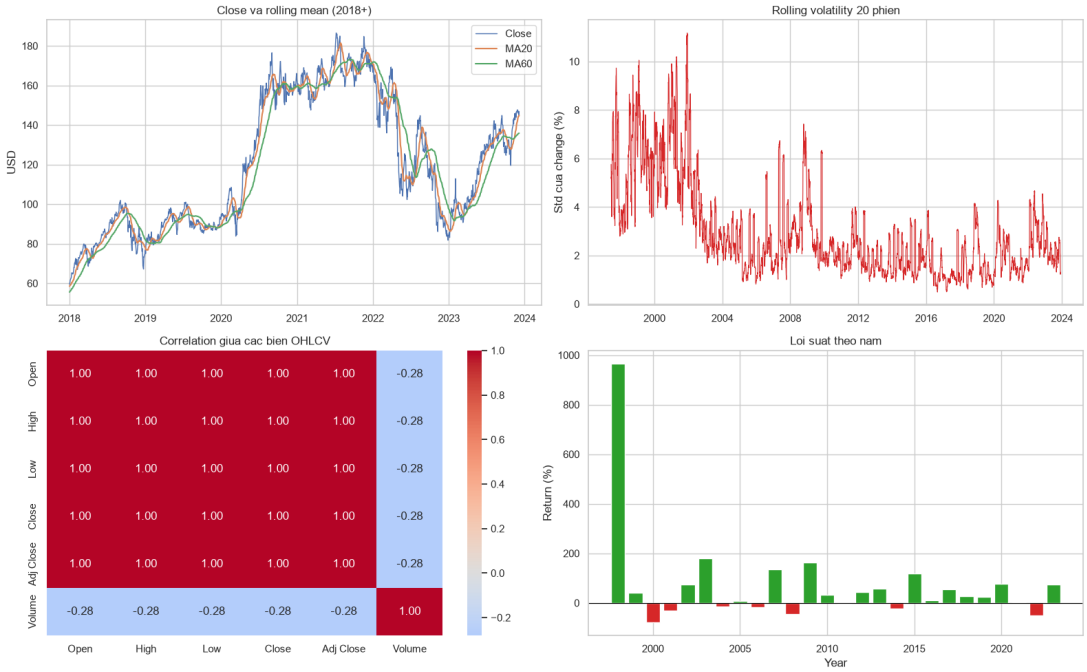
Tệp được dùng là `AMZN.csv` trong thư mục dữ liệu của Assignment 06; project không ghi lại URL tải chính xác của snapshot CSV này. Link ở trên là trang lịch sử giá AMZN để tham khảo, không được hiểu là bằng chứng về nguồn tải tệp. CSV có 6.684 phiên từ ngày 15/05/1997 đến 05/12/2023, với các cột Date, Open, High, Low, Close, Adj Close và Volume. Dòng đầu có Close bằng 0.097917 và Volume bằng 1.443.120.000; output của một cửa sổ 60 phiên là Close ở phiên kế tiếp. Trong bản CSV, Close thấp nhất khoảng 0.0698, trung vị 7.1048 và cao nhất 186.5705; khoảng biến thiên rất rộng phản ánh nhiều giai đoạn thị trường khác nhau.

Không có giá trị thiếu hay ngày trùng trong tệp Amazon. Các ngày cuối tuần và ngày nghỉ không có dòng; vì thế "bước tiếp theo" là phiên giao dịch kế tiếp, không phải ngày lịch tiếp theo. Các cột Open, High, Low, Close thường tương quan mạnh vì cùng một phiên. Notebook chỉ dùng Close trong mô hình RNN một biến; những biểu đồ OHLCV khác dùng cho khảo sát, không phải đầu vào của mô hình. Giá đóng cửa ở dạng mức có xu hướng dài hạn, còn phần trăm biến động ngày tập trung gần 0 và có đuôi dày. Điều này giải thích vì sao bài toán dự báo mức giá có thể khó ngoại suy khi test nằm ở vùng giá mới.

Sau lookback 60, notebook tạo 6.624 cửa sổ. StandardScaler fit trên phần train; mean khoảng 11.5805 và scale khoảng 17.4447. PyTorch train 107 epoch trong 101.50 giây; Keras



Hình 4.2: Chuỗi Close, phân phối mức giá, thay đổi ngày và volume trong dữ liệu Amazon.



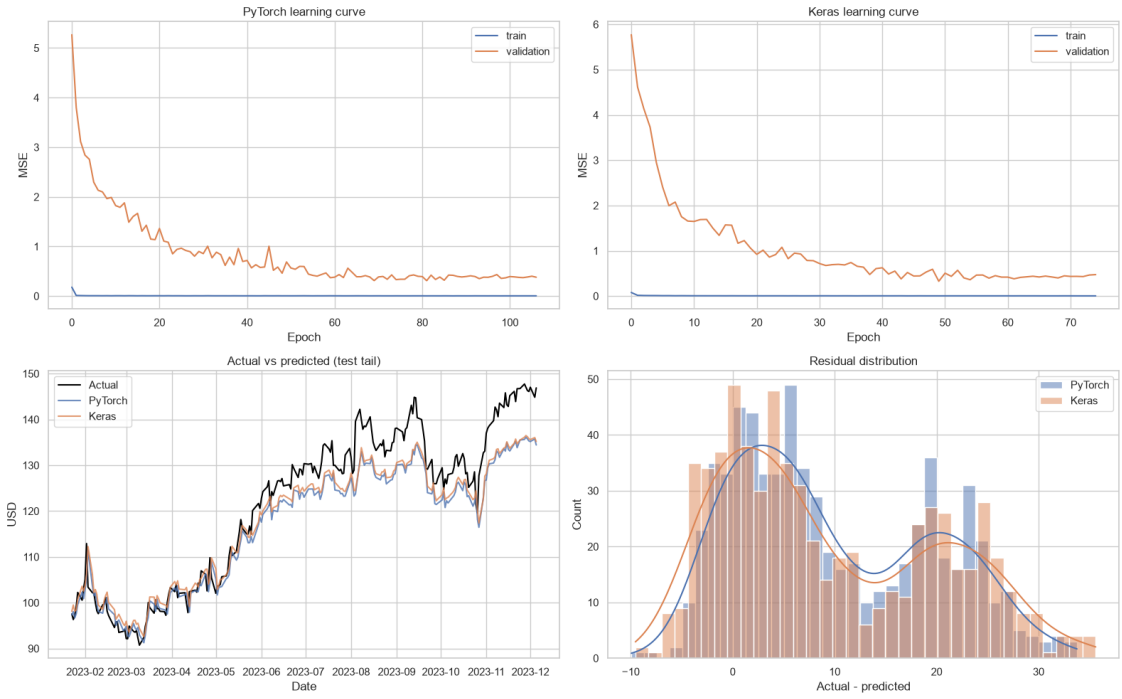
Hình 4.3: Rolling statistics, tương quan các cột OHLCV và thay đổi theo năm.

train 75 epoch trong 18.88 giây. Trọng số tốt nhất của từng framework được nạp lại trước khi tính metric trên test. Các chi tiết này cần được giữ khi tái lập, bởi nếu fit scaler trên toàn chuỗi hoặc lấy trạng thái epoch cuối, kết quả có thể khác đáng kể.

Bảng 4.2: Ba mô hình trên test Amazon Stock Price

Mô hình	MAE	RMSE	MAPE (%)	R^2	Train (s)
Persistence baseline	**2.2720**	**3.1507**	**1.7519**	**0.9874**	0.00
PyTorch RNN	10.4466	13.6738	6.7825	0.7633	101.50
Keras RNN	10.5704	14.1749	6.8377	0.7456	18.88

Persistence dẫn đầu trên cả bốn metric và cần được giữ lại như một kết quả chính của thí nghiệm. Với chuỗi mức giá biến đổi chậm, quy tắc lấy giá phiên gần nhất tận dụng trực tiếp tính liên tục giữa hai phiên. RNN nhỏ dễ co dự báo về vùng giá đã thấy trong train nên gặp khó khi phần test dịch sang mức cao hơn. PyTorch nhỉnh hơn Keras một chút về MAE, nhưng cả hai vẫn kém baseline rõ rệt. Các hướng như dự báo return/difference, rolling normalization hoặc walk-forward chỉ là đề xuất tiếp theo, chưa thuộc kết quả hiện tại.

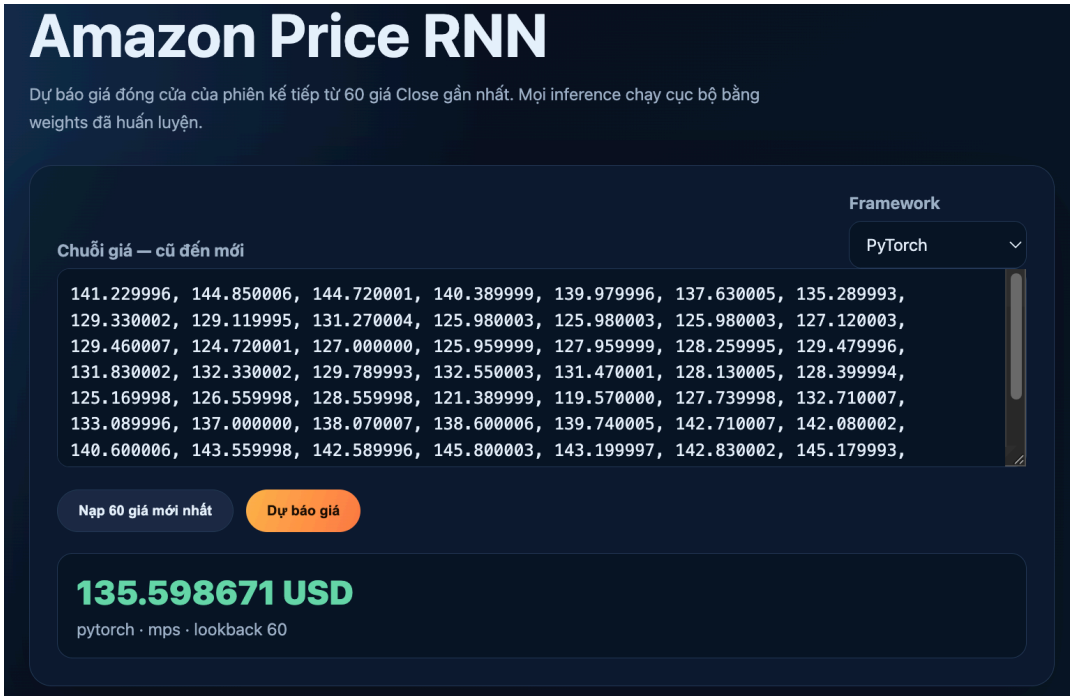


Hình 4.4: Learning curves, giá thực so với dự báo và residual trên test Amazon.

4.3 Avocado Price: nhiều chuỗi giá theo thị trường

Nguồn dữ liệu: `https://www.kaggle.com/datasets/neuromusic/avocado-prices`

Tệp `avocado.csv` có 18.249 dòng từ 04/01/2015 đến 25/03/2018. Mỗi dòng là thống kê một tuần cho một tổ hợp vùng và loại bơ, với target `AveragePrice`; các cột khác gồm sản



Hình 4.5: Giao diện deploy Amazon với chuỗi giá Close, framework và giá phiên kế tiếp dự đoán.

lượng, mã PLU, số lượng túi, `type`, năm và `region`. Dòng mẫu cho Albany, loại conventional vào 27/12/2015 có AveragePrice 1.33 và Total Volume 64.236,62. Trong CSV, giá thấp nhất 0.44, trung vị 1.37 và cao nhất 3.25. Hai loại conventional và organic có lần lượt 9.126 và 9.123 dòng, gần cân bằng về số quan sát.

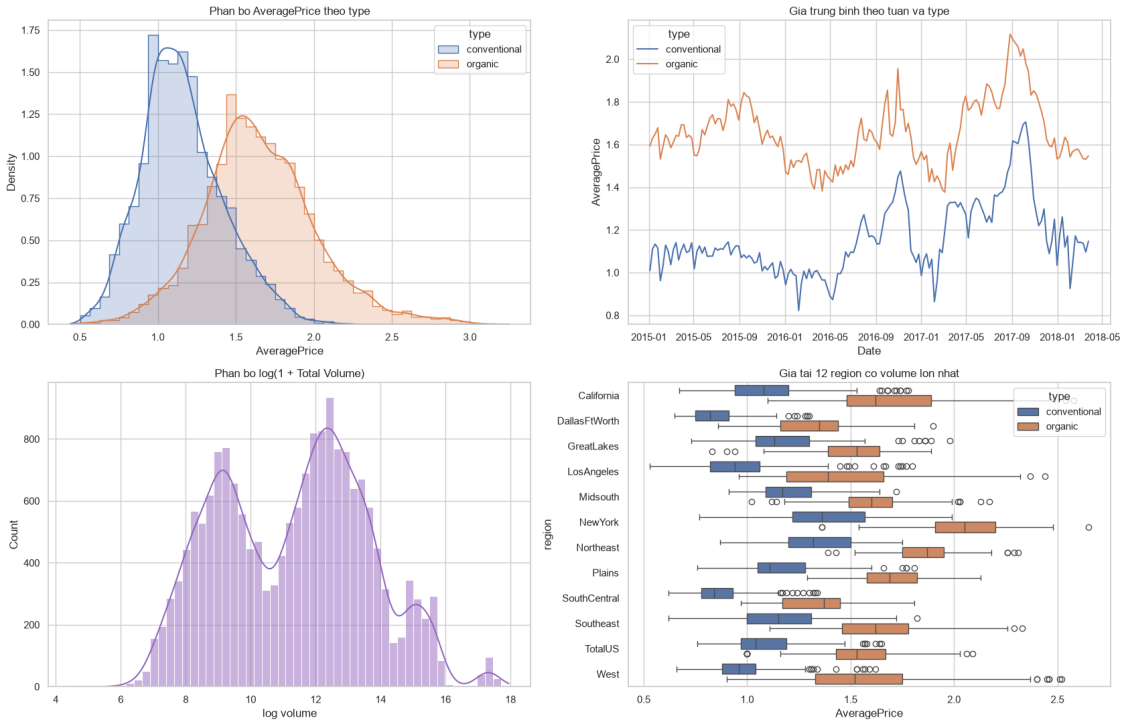
Khác Amazon, nhiều dòng có cùng ngày vì 54 region và hai type tạo thành 108 chuỗi độc lập. Do đó không được xếp toàn bộ CSV theo Date rồi tạo cửa sổ xuyên ranh giới region hoặc type. Notebook sắp xếp và tạo cửa sổ riêng trong mỗi nhóm, với lookback 12 tuần, sau đó gộp các cửa sổ để huấn luyện một mô hình chung. Scaler theo nhóm giữ mức giá riêng của từng thị trường. Input của một mẫu là 12 giá trung bình của cùng một region–type; output là AveragePrice tuần kế tiếp của đúng nhóm ấy.

Hai RNN có cùng kiến trúc với bài Amazon nhưng học trên nhiều chuỗi. PyTorch train 32 epoch trong 22.72 giây và Keras mất 10.88 giây. Kết quả tính trên 1.728 cửa sổ test sau khi đổi ngược về thang giá của từng nhóm. Đường học và đồ thị actual–predicted của một nhóm minh họa diễn biến, trong khi bảng metric tổng hợp là phép đánh giá trên toàn bộ các nhóm. Không nên lấy một nhóm tiêu biểu để suy ra chất lượng như nhau ở mọi vùng.

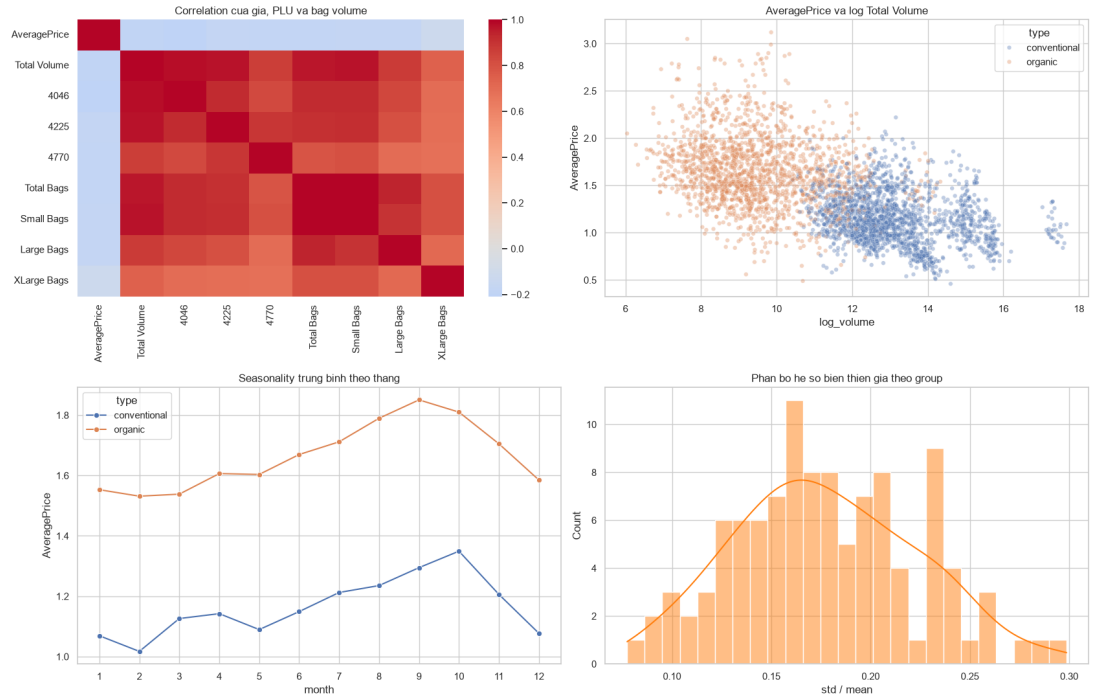
Bảng 4.3: Ba mô hình trên test Avocado Price

Mô hình	MAE	RMSE	MAPE (%)	R^2	Train (s)
Keras RNN	**0.1099**	**0.1534**	**8.7926**	**0.7490**	10.88
PyTorch RNN	0.1122	0.1562	9.0260	0.7398	22.72
Persistence baseline	0.1199	0.1742	9.5478	0.6762	0.00

Kết quả Avocado đối lập với Amazon: cả PyTorch và Keras RNN đều vượt persistence trên bốn metric. Keras tốt nhất với MAE 0.1099 và $R^2 = 0.7490$; chênh lệch giữa hai framework

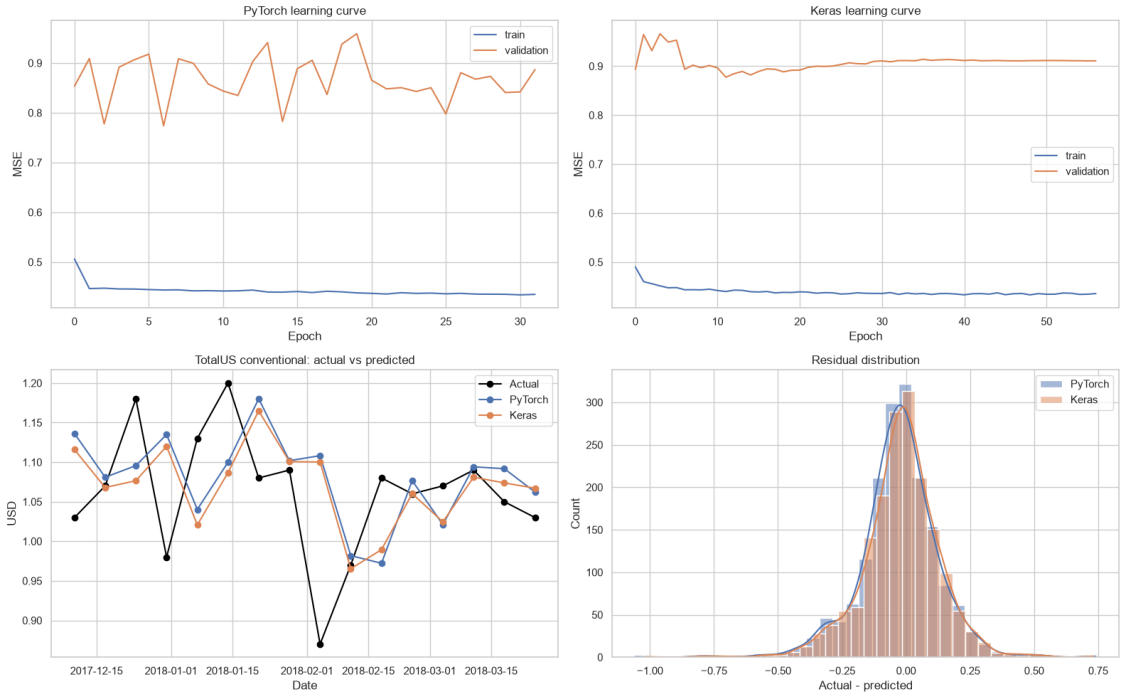


Hình 4.6: Tổng quan mức giá, sản lượng và khác biệt giữa các nhóm Avocado.



Hình 4.7: Mẫu hình mùa vụ, tương quan và biến động giá Avocado.

nhỏ hơn khoảng cách tới baseline theo RMSE. Có thể suy luận rằng dữ liệu giá tuần theo từng thị trường chứa mẫu hình hữu ích hơn quy tắc giữ nguyên giá trước đó, nhưng kết quả tổng hợp chưa chứng minh mức cải thiện giống nhau ở mọi region và type.



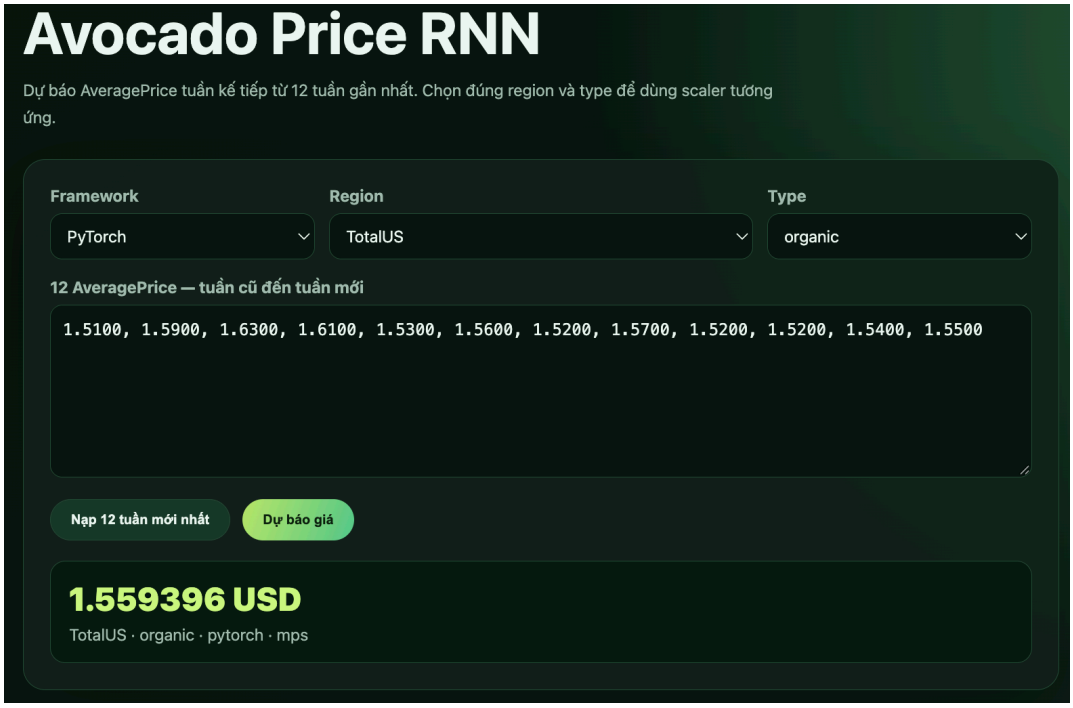
Hình 4.8: Đường học, actual–predicted và residual của Avocado trong notebook.

4.4 Kiểm tra luồng deploy trên localhost

Hai ứng dụng của Assignment 06 đã lưu weights PyTorch, Keras và metadata trong thư mục `models` của từng dataset. Người dùng không cần huấn luyện lại trước khi mở web. Tại localhost, ứng dụng Amazon mặc định dùng cổng 8001; Avocado dùng cổng 8002. Giao diện Amazon cho phép nạp 60 giá Close mới nhất từ CSV hoặc nhập 60 số theo thứ tự cũ đến mới. Giao diện Avocado cho phép chọn region, type rồi nạp 12 giá AveragePrice tuần gần nhất của đúng nhóm. Cả hai cho phép chọn PyTorch hoặc Keras trước khi nhấn nút dự báo.

Luồng inference cần giữ đúng thứ tự: kiểm tra đầu vào, chọn scaler từ metadata, chuẩn hóa chuỗi, nạp mô hình và checkpoint tương ứng, chạy dự báo rồi đổi ngược sang đơn vị giá. Việc nạp một bộ weights khác nhưng giữ scaler cũ có thể trả về một con số hợp lệ về mặt kiểu dữ liệu mà sai về ý nghĩa. Với Avocado, còn phải dùng scaler của region–type đã chọn. Đây là một điểm dễ lỗi khi chuyển code từ notebook sang ứng dụng web, vì notebook thường giữ sẵn các biến trong bộ nhớ, còn web phải tái tạo toàn bộ trạng thái từ artifact đã lưu.

Ứng dụng cần từ chối chuỗi không đủ độ dài, giá âm hoặc không hữu hạn, framework không được hỗ trợ và nhóm Avocado không có trong metadata. Việc từ chối dữ liệu xấu giúp người dùng hiểu lý do không có dự báo và ngăn dữ liệu sai đi vào mô hình. Tuy nhiên, một chuỗi có đúng 60 số dương vẫn có thể vô lý về kinh tế, ví dụ các giá nhập nhầm đơn vị hoặc đảo



Hình 4.9: Giao diện deploy Avocado với region, type, chuỗi giá, framework và kết quả dự đoán.

thứ tự thời gian. Kiểm tra đầu vào vì vậy giảm lỗi cơ học nhưng không đảm bảo giá trị dự báo có thể sử dụng.

Để tạo minh chứng deploy cho báo cáo, ảnh chụp nên được thực hiện sau khi nạp chuỗi và bấm dự báo. Ảnh Amazon phải thấy lựa chọn framework, 60 giá gần nhất hoặc phần nhập tương ứng và giá Close dự đoán. Ảnh Avocado phải thấy thêm region, type và 12 tuần giá; cả hai cần chụp cả kết quả xuất ra. Nếu ảnh chỉ hiển thị trang trống trước dự báo, nó chứng minh giao diện khởi động được nhưng chưa chứng minh mô hình đã chạy. Đó là lý do hai vị trí ảnh ở Hình 4.5 và Hình 4.9 được đặt sau phần kết quả benchmark.

4.5 Giới hạn của hai bài toán dự báo

Thứ nhất, cả hai notebook dự báo một bước, không đánh giá dự báo nhiều bước. Nếu cần giá sau bảy phiên hoặc bốn tuần, không thể lấy MAE một bước rồi suy ra sai số ở chân trời dài hơn. Một cách đơn giản là dự báo lặp: dùng kết quả bước trước làm đầu vào cho bước sau, nhưng lỗi sẽ tích lũy. Cách khác là huấn luyện mô hình dự báo trực tiếp từng chân trời. Hai cách này cần thiết kế tập train và baseline tương ứng, vì mục tiêu vận hành đã đổi.

Thứ hai, dữ liệu Amazon là một chuỗi đơn và chỉ dùng giá Close. Dữ liệu Avocado có nhiều chuỗi nhưng notebook cũng chỉ dùng giá quá khứ của cùng nhóm làm input. Các biến khác trong CSV, như Volume của Amazon hay Total Volume của Avocado, xuất hiện trong EDA chứ không đi vào RNN được báo cáo. Nếu bổ sung biến ngoại sinh, phải kiểm tra nó có thực sự biết được tại thời điểm dự báo hay không. Chẳng hạn doanh số tuần kế tiếp chưa thể dùng để dự báo giá tuần kế tiếp nếu doanh số ấy chỉ xuất hiện sau khi tuần kết thúc. Dùng biến không sẵn có lúc inference là một dạng rò rỉ dữ liệu dù metric test rất đẹp.

Thứ ba, metric tổng hợp trên Avocado trộn 108 chuỗi theo số cửa sổ. Nếu một số nhóm có nhiều quan sát hơn, chúng đóng góp nhiều hơn vào MAE chung. Để hiểu chất lượng theo thị trường, cần thêm bảng MAE theo region–type và xem phân phối lỗi giữa các nhóm. Một model chung có thể cải thiện trung bình nhưng kém baseline ở vài thị trường cụ thể. Tương tự, Amazon cần xem lỗi theo giai đoạn: một năm biến động mạnh có thể tạo phần lớn RMSE. Bảng tổng hợp ở tiểu luận phản ánh đúng notebook hiện có nhưng không trả lời hết những câu hỏi phân tầng này.

Thứ tư, dự báo giá không tương đương quyết định mua bán hay định giá giao dịch. Một mô hình có MAE thấp vẫn có thể không tạo lợi thế sau khi tính chi phí giao dịch và rủi ro; với Avocado, giá trung bình quan sát cũng không mô tả từng đơn hàng. Vì vậy phần ứng dụng nên trình bày kết quả như một ước lượng từ lịch sử và nêu phạm vi dữ liệu. Điểm mạnh của hai bài thực nghiệm là chúng buộc mô hình được đối chiếu với một quy tắc rất đơn giản. Khi RNN thua persistence trên Amazon, báo cáo giữ nguyên kết luận ấy; khi RNN thắng trên Avocado, báo cáo vẫn không suy rộng sang mọi chuỗi thời gian.

4.6 Tổng hợp Chương 4 và ý nghĩa triển khai

Hai dataset cho thấy baseline là thành phần không thể thiếu khi đánh giá dự báo chuỗi. Cùng dùng RNN một tầng 64 hidden units, Amazon không vượt quy tắc giữ giá phiên trước, trong khi Avocado cải thiện vừa phải so với baseline. Khác biệt đến từ cấu trúc dữ liệu, tần suất thời gian, phân bố train/test và cách tổ chức các nhóm chuỗi; vì thế không thể khẳng định RNN luôn tốt hay luôn kém. Thứ hạng framework cũng thay đổi theo dữ liệu: PyTorch tốt hơn một chút ở Amazon, còn Keras dẫn đầu trên Avocado.

Giao diện localhost hoàn thiện luồng từ cửa sổ đầu vào, lựa chọn framework, nạp trọng số đến hiển thị dự báo. Ảnh deploy vì vậy cần thể hiện cả input, mô hình và output để xác nhận quy trình end-to-end. Tuy nhiên, website trả về một con số hợp lệ chỉ chứng minh pipeline suy luận hoạt động; chất lượng vẫn phải dựa vào test theo thời gian và đối chiếu baseline. Khi dữ liệu mới vượt phạm vi CSV, hệ thống cần theo dõi sai số gần đây và xem xét huấn luyện lại.

Tài liệu tham khảo

[1] A. M. Turing, "Computing Machinery and Intelligence," *Mind*, 1950. https://academic.oup.com/mind/article/LIX/236/433/986238.

[2] J. McCarthy, M. Minsky, N. Rochester và C. Shannon, "A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence," 1955. https://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html.

[3] F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, 1958. https://doi.org/10.1037/h0042519.

[4] D. Rumelhart, G. Hinton và R. Williams, "Learning representations by back-propagating errors," *Nature*, 1986. https://doi.org/10.1038/323533a0.

[5] A. Krizhevsky, I. Sutskever và G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NeurIPS*, 2012. https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c-Abstract.html.

[6] A. Vaswani và cộng sự, "Attention Is All You Need," *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762.

[7] K. He và cộng sự, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[8] Y. Bengio, P. Simard và P. Frasconi, "Learning Long-Term Dependencies with Gradient Descent is Difficult," *IEEE Transactions on Neural Networks*, 1994. https://doi.org/10.1109/72.279181.

[9] A. Krizhevsky, V. Nair và G. Hinton, "CIFAR-10 and CIFAR-100 datasets." https://www.cs.toronto.edu/~kriz/cifar.html.

[10] D. T. Thắng, mã nguồn và báo cáo các Assignment của project *PTHTTM*. https://github.com/duongthang2710/PTHTTM.